FRE6871 R in Finance

Lecture#6, Fall 2025

Jerzy Pawlowski *jp3900@nyu.edu*

NYU Tandon School of Engineering

December 1, 2025

NYU | TANDON SCHOOL OF ENGINEERING

Trade and Quote (*TAQ*) Data

High frequency data is typically formatted as either Trade and Quote (*TAQ*) data, or *Open-High-Low-Close* (*OHLC*) data.

Trade and Quote (*TAQ*) data contains intraday *trades* and *quotes* on exchange-traded stocks and futures.

TAQ data is often called *tick data*, with a *tick* being a row of data containing new *trades* or *quotes*.

The *TAQ* data is spaced irregularly in time, with data recorded each time a new trade or quote arrives.

Each row of *TAQ* data may contain the quote and trade prices, and the corresponding quote size or trade volume: *Bid.Price, Bid.Size, Ask.Price, Ask.Size, Trade.Price, Volume*.

TAQ data is often split into *trade* data and *quote* data.

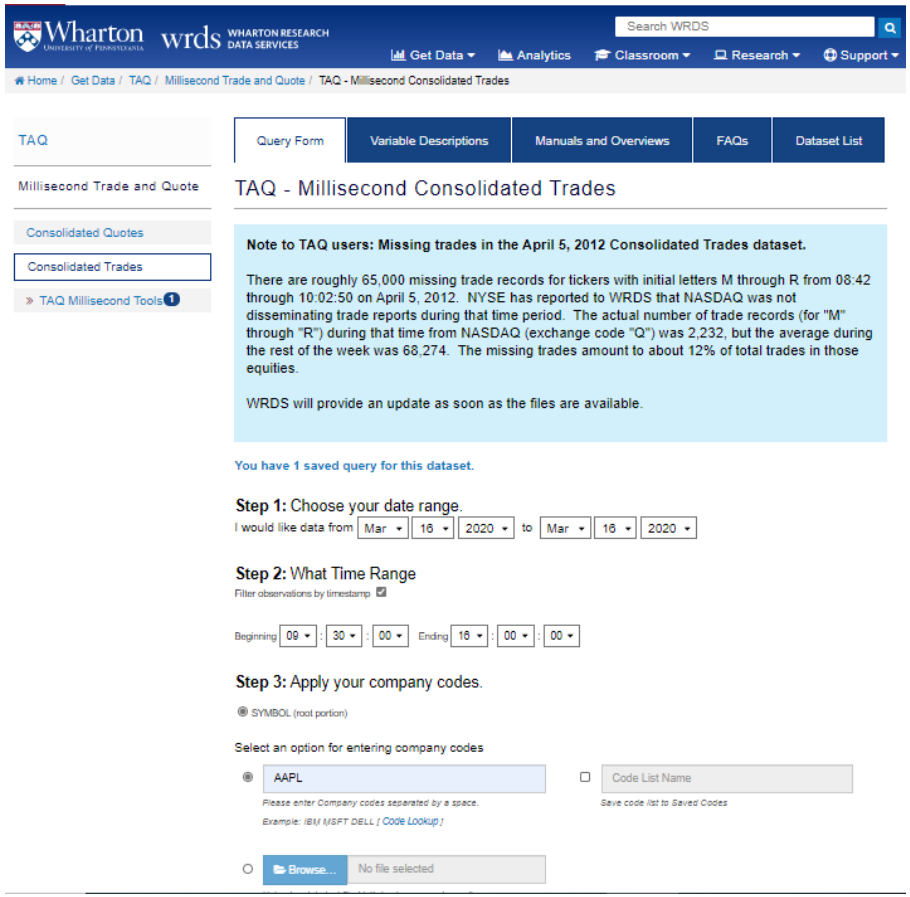
```
> # Load package HighFreq
> library(HighFreq)
> # Or load the high frequency data file directly:
> # symbolv <- load("/Users/jerzy/Develop/R/HighFreq/data/hf_data.RI
> head(HighFreq::SPY_TAQ)
> head(HighFreq::SPY)
> tail(HighFreq::SPY)
```

Downloading *TAQ* Data From *WRDS*

TAQ data can be downloaded from the *WRDS TAQ* web page.

The *TAQ* data are at millisecond frequency, and are *consolidated* (combined) from the New York Stock Exchange *NYSE* and other exchanges.

The *WRDS TAQ* web page provides separately *trades* data and separately *quotes* data.



Reading *TAQ* Data From .csv Files

Trade and Quote (*TAQ*) data stored in `.csv` files can be very large, so it's better to read it using the function `data.table::fread()` which is much faster than the function `read.csv()`.

Each *trade* or *quote* contributes a *tick* (row) of data, and the number of ticks can be very large (hundred of thousands per day, or more).

The function `strptime()` coerces `character` strings representing the date and time into `POSIXlt` *date-time* objects.

The argument `format="%H:%M:%OS"` allows the parsing of fractional seconds, for example `"15:59:59.989847074"`.

The function `as.POSIXct()` coerces objects into `POSIXct` *date-time* objects, with a `numeric` value representing the *moment of time* in seconds.

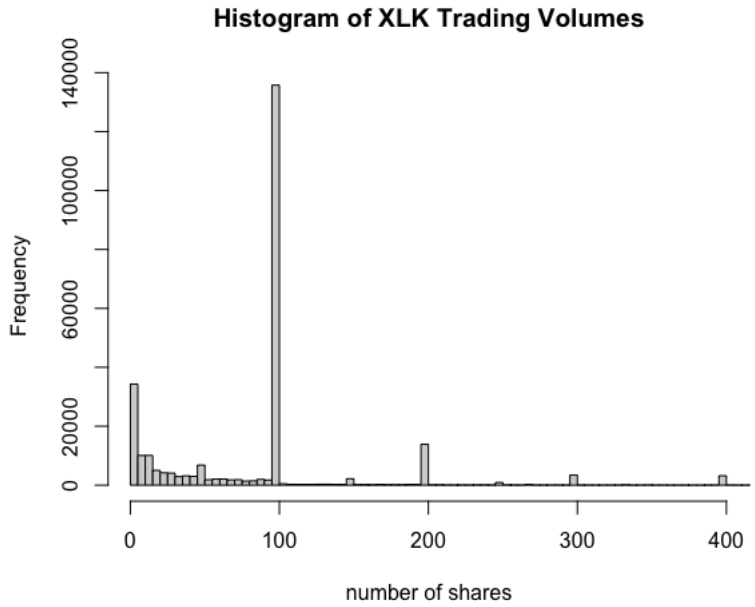
```
> library(HighFreq)
> # Read TAQ trade data from csv file
> taq <- data.table::fread(file="/Users/jerzy/Develop/lecture_slides
> # Inspect the TAQ data in data.table format
> taq
> class(taq)
> colnames(taq)
> sapply(taq, class)
> symboln <- taq$SYM_ROOT[1]
> # Create date-time index
> datev <- paste(taq$DATE, taq$TIME_M)
> # Coerce date-time index to POSIXlt
> datev <- strptime(datev, "%Y%m%d %H:%M:%OS")
> class(datev)
> # Display more significant digits
> # options("digits")
> options(digits=20, digits.secs=10)
> last(datev)
> unclass(last(datev))
> as.numeric(last(datev))
> # Coerce date-time index to POSIXct
> datev <- as.POSIXct(datev)
> class(datev)
> last(datev)
> unclass(last(datev))
> as.numeric(last(datev))
> # Calculate the number of seconds
> as.numeric(last(datev)) - as.numeric(first(datev))
> # Calculate the number of ticks per second
> NROW(taq)/(6.5*3600)
> # Select TAQ data columns
> taq <- taq[, .(price=PRICE, volume=SIZE)]
```

Trading Volumes in High Frequency Data

The trading volumes represent the number of shares traded at a given price.

The histogram of the trading volumes shows that the highest frequencies of trades are for *round lots* - trades that are multiples of 100 shares.

There are also significant frequencies for *odd lots*, with small volumes of less than 100 shares.




```
> # Coerce trade ticks to xts series
> xlk <- xts::xts(taq[, .(price, volume)], datev)
> colnames(xlk) <- c("price", "volume")
> save(xlk, file="/Users/jerzy/Develop/data/xlk_tick_trades_2020031
> # Plot histogram of the trading volumes
> hist(xlk$volume, main="Histogram of XLK Trading Volumes",
+      breaks=1e5, xlim=c(1, 400), xlab="number of shares")
```

Microstructure Noise in High Frequency Data

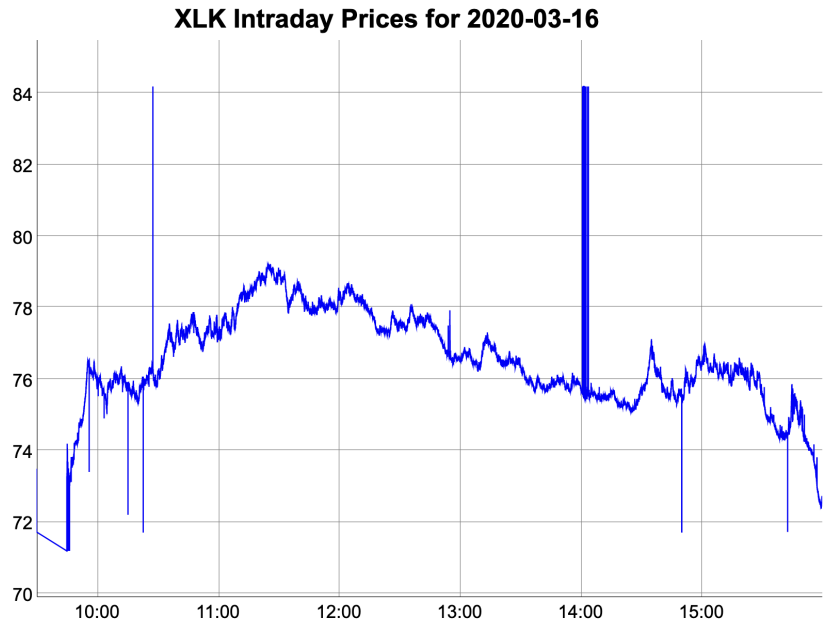
High frequency data contains *microstructure noise* in the form of *price spikes* and the *bid-ask bounce*.

Price spikes are single ticks with prices far away from the average.

Price spikes are often caused by data collection errors, but sometimes they represent actual trades with very large lot (trade) sizes.

The *bid-ask bounce* is the bouncing of traded prices between the bid and ask prices.

The *bid-ask bounce* creates an illusion of rapidly changing prices, while in reality the mid price is unchanged.




```
> # Plot dygraph
> dygraphs::dygraph(xlk$price, main="XLK Intraday Prices for 2020-03
+   dyOptions(colors="blue", strokeWidth=1)
> # Plot in x11 window
> x11(width=6, height=5)
> quantmod::chart_Series(x=xlk$price, name="XLK Intraday Prices for
```

The Bid-ask Bounce of High Frequency Prices

The *bid-ask bounce* is the bouncing of traded prices between the bid and ask prices.

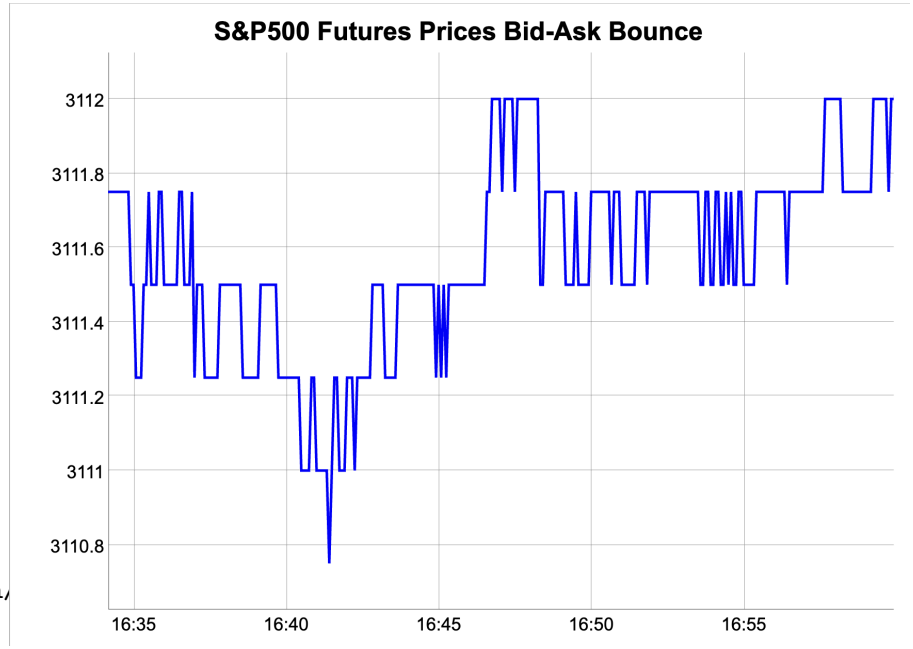
The *bid-ask bounce* is prominent at very high frequency time scales or in periods of low volatility.

The *bid-ask bounce* creates an illusion of rapidly changing prices, while in fact the mid price is constant.

The *bid-ask bounce* inflates the estimates of realized volatility, above the actual volatility.

The *bid-ask bounce* creates the appearance of mean reversion (negative autocorrelation), that isn't tradeable for most traders.

```
> pricev <- read.zoo(file="/Users/jerzy/Develop/lecture_slides/data/
+   header=TRUE, sep=",")
> pricev <- as.xts(pricev)
> dygraphs::dygraph(pricev$Close,
+   main="S&P500 Futures Prices Bid-Ask Bounce") %>%
+   dyOptions(colors="blue", strokeWidth=2)
```



Price Spikes And Trading Volumes in High Frequency Data

The number of the *price spikes* depends on the level of trading volumes, with the number decreasing with higher trading volumes.

The number of price spikes is lower for trade prices with larger trading volumes.


```
> # Plot dygraph of trade prices of at least 100 shares
> dygraphs::dygraph(xlk$price[xlk$volume >= 100, ],
+   main="XLK Prices for Trades of At Least 100 Shares") %>%
+   dyOptions(colors="blue", strokeWidth=1)
```

Removing Odd Lot Trades From *TAQ* Data

The trading volumes represent the number of shares traded at a given price.

The histogram of the trading volumes shows that the highest frequencies of trades are for *round lots* - trades that are multiples of 100 shares.

There are also significant frequencies for *odd lots*, with small volumes of less than 100 shares.

The *odd lot* ticks are often removed to reduce the size of the *TAQ* data.

Selecting only the large lot trades reduces microstructure noise (price spikes, bid-ask bounce) in high frequency data.


```
> # Select the large trade lots of at least 100 shares
> dim(taq)
> tickb <- taq[taq$volume >= 100]
> dim(tickb)
> # Number of large lot ticks per second
> NROW(tickb)/(6.5*3600)
> # Plot histogram of the trading volumes
> hist(tickb$volume, main="Histogram of XLK Trading Volumes",
+      breaks=100000, xlim=c(1, 400), xlab="number of shares")
> # Save trade ticks with large lots
> data.table::fwrite(tickb, file="/Users/jerzy/Develop/data/xlk_tick_trades_20200316_biglots.csv")
> # Coerce trade prices to xts
> xlkb <- xts::xts(tickb[, .(price, volume)], tickb$index)
> colnames(xlkb) <- c("price", "volume")
```

```
> # Plot dygraph of the large lots
> dygraphs::dygraph(xlkb$price,
+   main="XLK Prices for Trades of At Least 100 Shares") %>%
+   dyOptions(colors="blue", strokeWidth=1)
> # Plot the large lots
> x11(width=6, height=5)
> quantmod::chart_Series(x=xlk$price,
+   name="XLK Trade Ticks for 2020-03-16 (large lots only)")
```

Aggregating *TAQ* Data to *OHLC*

The *data table* columns can be *aggregated* over categories (factors) defined by one or more columns passed to the "by" argument.

Multiple *data table* columns can be referenced by passing a list of names specified by the dot `.()` operator.

The function `round.POSIXt()` rounds date-time objects to seconds, minutes, hours, days, months or years.

The function `as.POSIXct()` coerces objects to class `POSIXct`.

```
> # Round time index to seconds
> tickg[, zoo::index := as.POSIXct(round.POSIXt(index, "secs"))]
> # Aggregate to OHLC by seconds
> ohlc <- tickg[, .(open=first(price), high=max(price), low=min(pri
> # Round time index to minutes
> tickg[, zoo::index := as.POSIXct(round.POSIXt(index, "mins"))]
> # Aggregate to OHLC by minutes
> ohlc <- tickg[, .(open=first(price), high=max(price), low=min(price), close=last(price), volume=sum(volume)), by=index]
```


```
> # Coerce OHLC prices to xts
> ohlc <- xts::xts(ohlc[, -"index"], ohlc$index)
> # Plot dygraph of the OHLC prices
> dygraphs::dygraph(ohlc[, -5], main="XLK Trade Ticks for 2020-03-16
+   dyCandlestick()
> # Plot the OHLC prices
> x11(width=6, height=5)
> quantmod::chart_Series(x=ohlc, TA="add_Vo()",
+   name="XLK Trade Ticks for 2020-03-16 (OHLC)")
```

Open-High-Low-Close (*OHLC*) Data

Open-High-Low-Close (*OHLC*) data contains intraday trade prices and trade volumes.

OHLC data is evenly spaced in time, with each row containing the *Open, High, Low, Close* prices, and the trade *Volume*, recorded over the past time interval (called a *bar* of data).

The *Open* and *Close* prices are the first and last trade prices recorded in the time bar.

The *High* and *Low* prices are the highest and lowest trade prices recorded in the time bar.

The *Volume* is the total trading volume recorded in the time bar.

The *OHLC* data format provides a way of efficiently compressing *TAQ* data, while preserving information about price levels, volatility (range), and trading volumes.

In addition, evenly spaced *OHLC* data allows for easier analysis of multiple time series, since the prices for different assets are given at the same moments in time.

```
> # Load package HighFreq
> library(HighFreq)
> head(HighFreq::SPY)
                    SPY.Open SPY.High SPY.Low SPY.Close SPY.Volume
2008-01-02 09:31:00      147      147     147       147     591203
2008-01-02 09:32:00      147      147     147       147     385457
2008-01-02 09:33:00      147      147     147       147     343700
2008-01-02 09:34:00      147      147     147       147     863418
2008-01-02 09:35:00      147      147     147       147     457500
2008-01-02 09:36:00      147      147     147       147     416708
```

Plotting High Frequency *OHLC* Data

Aggregating high frequency *TAQ* data into *OHLC* format with lower periodicity allows for data compression while maintaining some information about volatility.

```
> # Load package HighFreq
> library(HighFreq)
> # Define symbol
> symboln <- "SPY"
> # Load OHLC data
> dirout <- "/Users/jerzy/Develop/data/minutes/"
> symboln <- load(file.path(dirout, paste0(symboln, ".RData")))
> interval <-"2013-11-11 09:30:00/2013-11-11 10:30:00"
> chart_Series(SPY[interval], name=symboln)
```

The package *HighFreq* contains both *TAQ* data and *Open-High-Low-Close* (*OHLC*) data.

If you are not able to install package *HighFreq* then download the file `hf_data.RData` from the NYU share drive and load it.

Package *HighFreq* for Managing High Frequency Data

The package *HighFreq* contains functions for managing high frequency time series data, such as:

- converting *TAQ* data to *OHLC* format,
- chaining and joining time series,
- scrubbing bad data,
- managing time zones and alligning time indices,
- aggregating data to lower frequency (periodicity),
- calculating rolling aggregations (VWAP, Hurst exponent, etc.),
- calculating seasonality aggregations,
- estimating volatility, skewness, and higher moments,

```
> # Install package HighFreq from github
> devtools::install_github(repo="algoquant/HighFreq")
> # Load package HighFreq
> library(HighFreq)
> # Get documentation for package HighFreq
> # Get short description
> packageDescription(HighFreq)
> # Load help page
> help(package=HighFreq)
> # List all datasets in HighFreq
> data(package=HighFreq)
> # List all objects in HighFreq
> ls("package:HighFreq")
> # Remove HighFreq from search path
> detach("package:HighFreq")
```

Datasets in Package *HighFreq*

The package *HighFreq* contains several high frequency time series, in *xts* format, stored in a file called `hf_data.RData`:

- a time series called `SPY_TAQ`, containing a single day of *TAQ* data for the *SPY* ETF.
- three time series called `SPY`, `TLT`, and `VXX`, containing intraday 1-minute *OHLC* price bars for the *SPY*, *TLT*, and *VXX* ETFs.

Even after the *HighFreq* package is loaded, its datasets aren't loaded into the workspace, so they aren't listed in the workspace.

That's because the datasets in package *HighFreq* are set up for *lazy loading*, which means they can be called as if they were loaded, even though they're not loaded into the workspace.

The datasets in package *HighFreq* can be loaded into the workspace using the function `data()`.

The data is set up for *lazy loading*, so it doesn't require calling `data(hf_data)` to load it into the workspace before calling it.

```
> # Load package HighFreq
> library(HighFreq)
> # You can see SPY when listing objects in HighFreq
> ls("package:HighFreq")
> # You can see SPY when listing datasets in HighFreq
> data(package=HighFreq)
> # But the SPY dataset isn't listed in the workspace
> ls()
> # HighFreq datasets are lazy loaded and available when needed
> head(HighFreq::SPY)
> # Load all the datasets in package HighFreq
> data(hf_data)
> # HighFreq datasets are now loaded and in the workspace
> head(HighFreq::SPY)
```

Distribution of High Frequency Returns

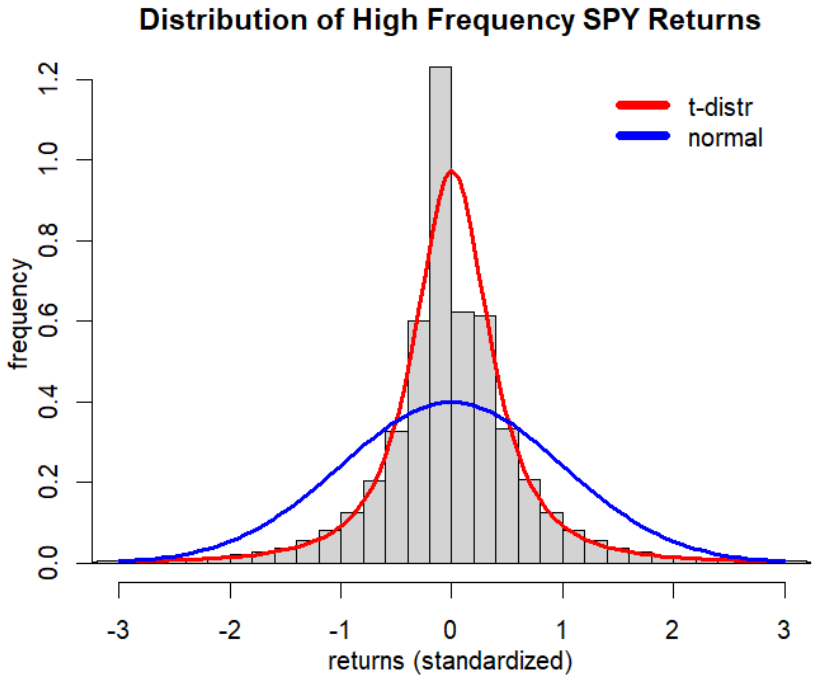
High frequency returns exhibit *large negative skewness* and *very large kurtosis* (leptokurtosis), or fat tails.

Student's *t-distribution* has fat tails, so it fits high frequency returns much better than the normal distribution.

The function `fitdistr()` from package *MASS* fits a univariate distribution into a sample of data, by performing *maximum likelihood* optimization.

The function `hist()` calculates and plots a histogram, and returns its data *invisibly*.

```
> # Calculate SPY percentage returns
> ohlc <- HighFreq::SPY
> nrows <- NROW(ohlc)
> closep <- log(quantmod::Cl(ohlc))
> retp <- rutils::diffit(closep)
> colnames(retp) <- "SPY"
> # Standardize raw returns to make later comparisons
> retp <- (retp - mean(retp))/sd(retp)
> # Calculate moments and perform normality test
> sapply(c(var=2, skew=3, kurt=4), function(x) sum(retp^x)/nrows)
> tseries::jarque.bera.test(retp)
> # Fit SPY returns using MASS::fitdistr()
> optiml <- MASS::fitdistr(retp, densfun="t", df=2)
> loc <- optiml$estimate[1]
> scalev <- optiml$estimate[2]
```




```
> # Plot histogram of SPY returns
> histp <- hist(retp, col="lightgrey", mgp=c(2, 1, 0),
+   xlab="returns (standardized)", ylab="frequency", xlim=c(-3, 3),
+   breaks=1e3, freq=FALSE, main="Distribution of High Frequency SPY
> # lines(density(retp, bw=0.2), lwd=3, col="blue")
> # Plot t-distribution function
> curve(expr=dt((x-loc)/scalev, df=2)/scalev,
+ type="l", lwd=3, col="red", add=TRUE)
> # Plot the Normal probability distribution
> curve(expr=dnorm(x, mean=mean(retp),
+   sd=sd(retp)), add=TRUE, lwd=3, col="blue")
> # Add legend
> legend("topright", inset=0.05, bty="n",
+   leg=c("t-distr", "normal"), y.intersp=0.5,
+   lwd=6, lty=1, col=c("red", "blue"))
```

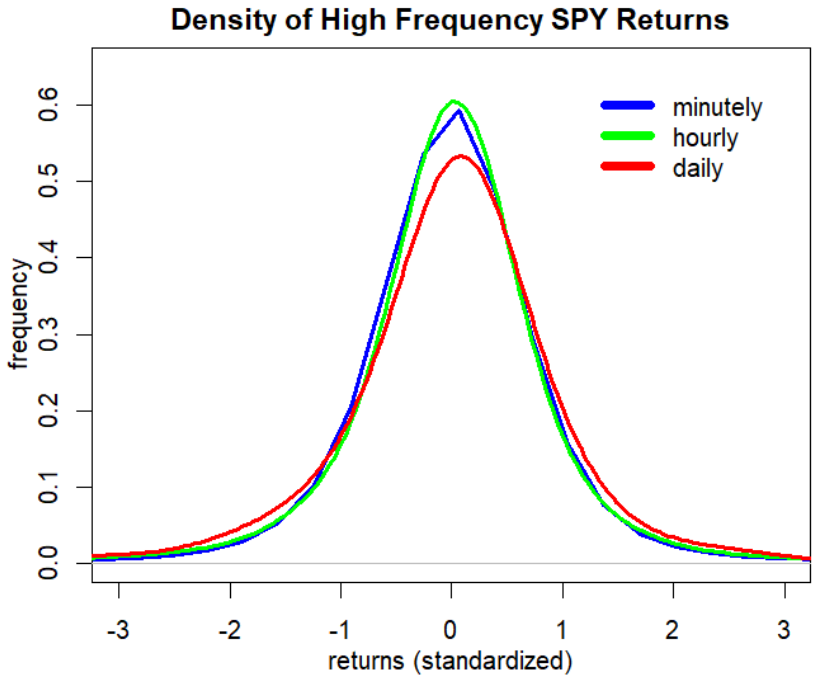
Distribution of Aggregated High Frequency Returns

The distribution of returns depends on the sampling frequency.

High frequency returns aggregated to a lower periodicity become less negatively skewed and less fat tailed, and closer to the normal distribution.

The function `xts::to.period()` converts a time series to a lower periodicity (for example from hourly to daily periodicity).

```
> # Hourly SPY percentage returns
> closep <- log(Cl(xts::to.period(x=ohlc, period="hours")))
> retsh <- rutils::diffit(closep)
> retsh <- (retsh - mean(retsh))/sd(retsh)
> # Daily SPY percentage returns
> closep <- log(Cl(xts::to.period(x=ohlc, period="days")))
> retd <- rutils::diffit(closep)
> retd <- (retd - mean(retd))/sd(retd)
> # Calculate moments
> sapply(list(minutely=retp, hourly=retsh, daily=retd),
+  function(rets) {sapply(c(var=2, skew=3, kurt=4),
+           function(x) mean(rets^x))
+ })  ##  end sapply
```



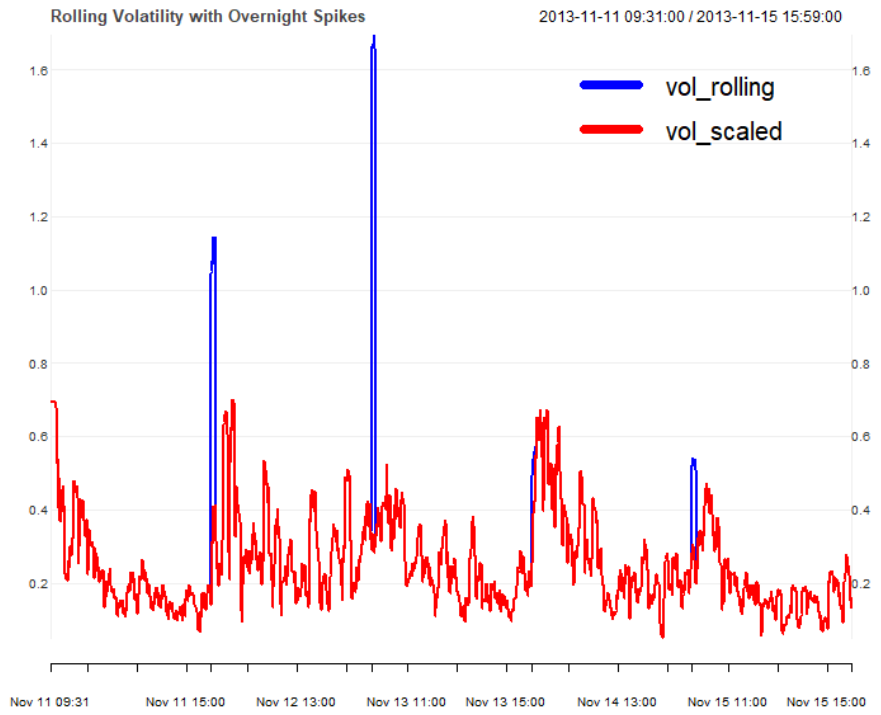

```
> # Plot densities of SPY returns
> plot(density(retp, bw=0.4), xlim=c(-3, 3),
+      lwd=3, mgp=c(2, 1, 0), col="blue",
+      xlab="returns (standardized)", ylab="frequency",
+      main="Density of High Frequency SPY Returns")
> lines(density(retsh, bw=0.4), lwd=3, col="green")
> lines(density(retd, bw=0.4), lwd=3, col="red")
> # Add legend
> legend("topright", inset=0.05, bty="n",
+   leg=c("minutely", "hourly", "daily"), y.intersp=0.5,
+   lwd=6, lty=1, col=c("blue", "green", "red"))
```

Estimating Rolling Volatility of High Frequency Returns

The volatility of high frequency returns can be inflated by large overnight returns.

The large overnight returns can be scaled down by dividing them by the overnight time interval.

```
> # Calculate rolling volatility of SPY returns
> ret2013 <- retp["2013-11-11/2013-11-15"]
> # Calculate rolling volatility
> lookb <- 11 ##  Look-back interval
> endd <- seq_along(ret2013)
> startp <- c(rep_len(1, lookb),
+   endd[1:(NROW(endd)-lookb)])
> endd[endd < lookb] <- lookb
> vol_rolling <- sapply(seq_along(endd),
+   function(it) sd(ret2013[startp[it]:endd[it]]))
> vol_rolling <- xts::xts(vol_rolling, zoo::index(ret2013))
> # Extract time intervals of SPY returns
> indeks <- c(60, diff(xts::.index(ret2013)))
> head(indeks)
> table(indeks)
> # Scale SPY returns by time intervals
> ret2013 <- 60*ret2013/indeks
> # Calculate scaled rolling volatility
> vol_scaled <- sapply(seq_along(endd),
+   function(it) sd(ret2013[startp[it]:endd[it]]))
> vol_rolling <- cbind(vol_rolling, vol_scaled)
> vol_rolling <- na.omit(vol_rolling)
> sum(is.na(vol_rolling))
> sapply(vol_rolling, range)
```




```
> # Plot rolling volatility
> x11(width=6, height=5)
> themev <- chart_theme()
> themev$col$line.col <- c("blue", "red")
> chart_Series(vol_rolling, theme=themev,
+      name="Rolling Volatility with Overnight Spikes")
> legend("topright", legend=colnames(vol_rolling),
+   inset=0.1, bg="white", lty=1, lwd=6, y.intersp=0.5,
+   col=themev$col$line.col, bty="n")
```

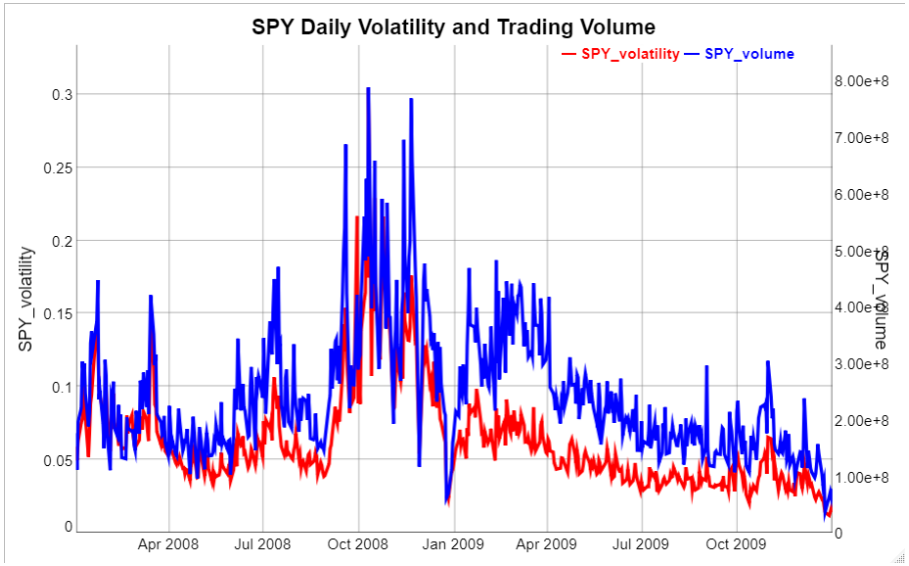
Daily Volume and Volatility of High Frequency Returns

Trading volumes typically rise together with market price volatility.

The function `apply.daily()` from package `xts` applies functions to time series over daily periods.

The function `calc_var_ohlc()` from package `HighFreq` calculates the variance of an *OHLC* time series using range estimators.

```
> # Volatility of SPY
> sqrt(HighFreq::calcvar_ohlc(ohlc))
> # Daily SPY volatility and volume
> volatd <- sqrt(xts::apply.daily(ohlc, FUN=calcvar_ohlc))
> colnames(volatd) <- ("SPY_volatility")
> volumv <- quantmod::Vo(ohlc)
> volumd <- xts::apply.daily(volumv, FUN=sum)
> colnames(volumd) <- ("SPY_volume")
> # Plot SPY volatility and volume
> datav <- cbind(volatd, volumd)["2008/2009"]
> colv <- colnames(datav)
> dygraphs::dygraph(datav,
+   main="SPY Daily Volatility and Trading Volume") %>%
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", col="red", strokeWidth=3) %>%
+   dySeries(name=colv[2], axis="y2", col="blue", strokeWidth=3)
```

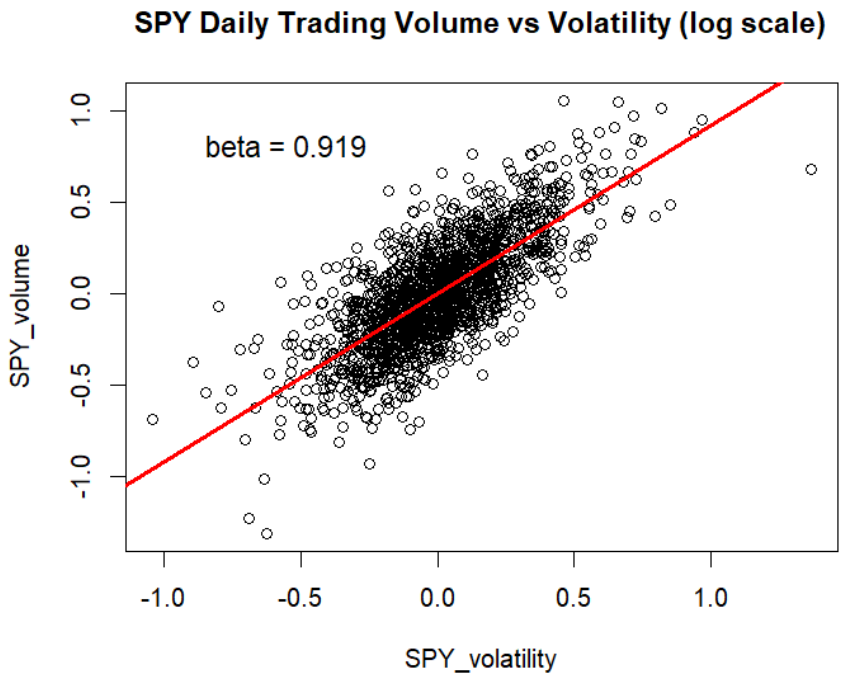


Beta of Volume vs Volatility of High Frequency Returns

As a general empirical rule, the *trading volume* υ in a given time period is roughly proportional to the *volatility* of the returns σ: $\upsilon \propto \sigma$.

The regression of the *log trading volume* versus the *log volatility* fails the *Durbin-Watson test* for the autocorrelation of residuals.

But the regression of the *differences* passes the *Durbin-Watson test*.

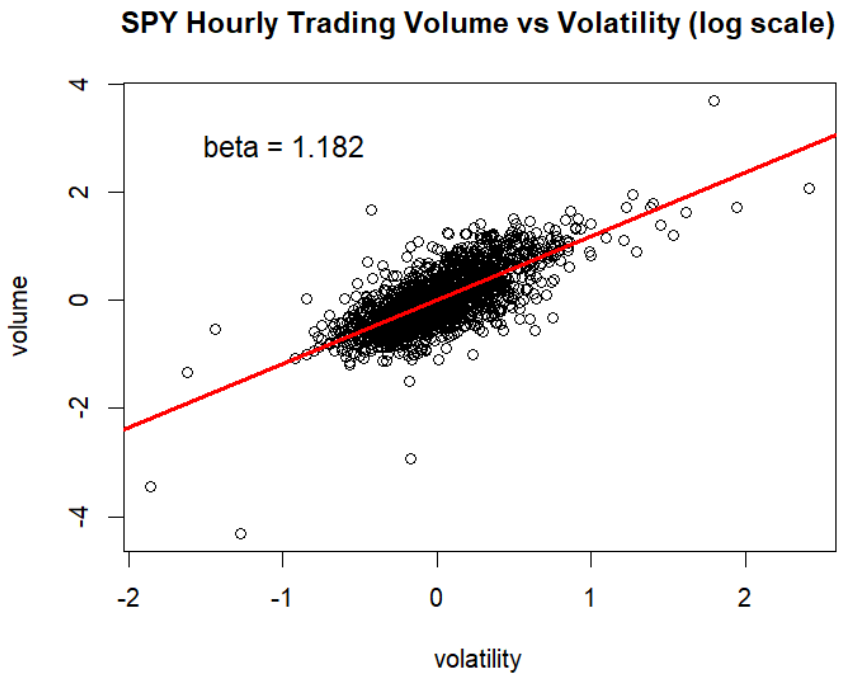



```
> # Regress log of daily volume vs volatility
> datav <- log(cbind(volumd, volatd))
> colv <- colnames(datav)
> dframe <- as.data.frame(datav)
> formulav <- as.formula(paste(colv, collapse="~"))
> regmod <- lm(formulav, data=dframe)
> # Durbin-Watson test for autocorrelation of residuals
> lmtest::dwtest(regmod)
> # Regress diff log of daily volume vs volatility
> dframe <- as.data.frame(rutils::diffit(datav))
> regmod <- lm(formulav, data=dframe)
> lmtest::dwtest(regmod)
> summary(regmod)
> plot(formulav, data=dframe, main="SPY Daily Trading Volume vs Volatility (log scale)")
> abline(regmod, lwd=3, col="red")
> mtext(paste("beta =", round(coef(regmod)[2], 3)), cex=1.2, lwd=3, side=2, las=2, adj=(-0.5), padj=(-7))
```

Hourly Trading Volume vs Volatility

Hourly aggregations of high frequency data also support the rule that the *trading volume* is roughly proportional to the *volatility* of the returns: $\upsilon \propto \sigma$.

```
> # 60 minutes of data in lookb interval
> lookb <- 60 ##  Look-back interval
> vol2013 <- volumv["2013"]
> ret2013 <- retp["2013"]
> # Define end points with beginning stub
> nrows <- NROW(ret2013)
> nagg <- nrows %/% lookb
> endd <- nrows-lookb*nagg + (0:nagg)*lookb
> startp <- c(1, endd[1:(NROW(endd)-1)])
> # Calculate SPY volatility and volume
> datav <- sapply(seq_along(endd), function(it) {
+   endp <- startp[it]:endd[it]
+   c(volume=sum(vol2013[endp]),
+     volatility=sd(ret2013[endp]))
+ })  ##  end sapply
> datav <- t(datav)
> datav <- rutils::diffit(log(datav))
> dframe <- as.data.frame(datav)
```




```
> formulav <- as.formula(paste(colnames(datav), collapse="~"))
> regmod <- lm(formulav, data=dframe)
> lmtest::dwtest(regmod)
> summary(regmod)
> plot(formulav, data=dframe,
+      main="SPY Hourly Trading Volume vs Volatility (log scale)")
> abline(regmod, lwd=3, col="red")
> mtext(paste("beta =", round(coef(regmod)[2], 3)), cex=1.2, lwd=3,
```

High Frequency Returns in Trading Time

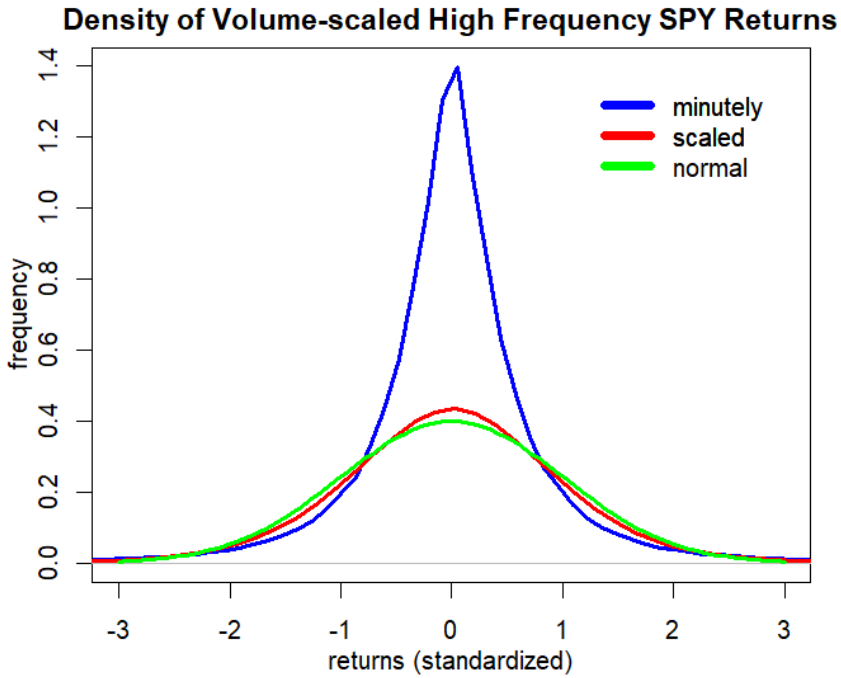
The *trading time* (volume clock) is the time measured by the level of *trading volume*, with the *volume clock* running faster in periods of higher *trading volume*.

The time-dependent volatility of high frequency returns (*heteroskedasticity*) produces their *leptokurtosis* (large kurtosis, or fat tails).

The returns can be divided by the *square root of the trading volumes* to obtain scaled returns over equal trading volumes.

But the returns should not be divided by very small volumes below a certain threshold.

The scaled returns have a smaller *skewness* and *kurtosis*, and they also have even higher autocorrelations than unscaled returns.




```
> # Scale returns using volume (volume clock)
> retsc <- ifelse(volumv > 1e4, retp/sqrt(volumv), 0)
> retsc <- retsc/sd(retsc)
> # Calculate moments of scaled returns
> nrows <- NROW(retp)
> sapply(list(retp=retp, retsc=retsc),
+   function(rets) {sapply(c(skew=3, kurt=4),
+     function(x) sum((rets/sd(rets))^x)/nrows)
+ })  ##  end sapply
```

```
> x11(width=6, height=5)
> par(mar=c(3, 3, 2, 1), oma=c(1, 1, 1, 1))
> # Plot densities of SPY returns
> plot(density(retp), xlim=c(-3, 3),
+      lwd=3, mgp=c(2, 1, 0), col="blue",
+      xlab="returns (standardized)", ylab="frequency",
+      main="Density of Volume-scaled High Frequency SPY Returns")
> lines(density(retsc, bw=0.4), lwd=3, col="red")
> curve(expr=dnorm, add=TRUE, lwd=3, col="green")
> # Add legend
> legend("topright", inset=0.05, bty="n", y.intersp=0.5,
+   leg=c("minutely", "scaled", "normal"),
+   lwd=6, lty=1, col=c("blue", "red", "green"))
```

Autocorrelations of High Frequency Returns

The *Ljung-Box* test, tests if the autocorrelations of a time series are *statistically significant*.

The *null hypothesis* of the *Ljung-Box* test is that the autocorrelations are equal to zero.

The *Ljung-Box* statistic is small for time series that have *statistically insignificant* autocorrelations.

The function `Box.test()` calculates the *Ljung-Box* test and returns the test statistic and its p-value.

For *minutely SPY* returns, the *Ljung-Box* statistic is large and its *p*-value is very small, so we can conclude that *minutely SPY* returns have statistically significant autocorrelations.

For *scaled minutely SPY* returns, the *Ljung-Box* statistic is even larger, so its autocorrelations are even more statistically significant.

SPY returns aggregated to longer time intervals are less autocorrelated.

```
> # Ljung-Box test for minutely SPY returns
> Box.test(retp, lag=10, type="Ljung")
> # Ljung-Box test for daily SPY returns
> Box.test(retd, lag=10, type="Ljung")
> # Ljung-Box test statistics for scaled SPY returns
> sapply(list(retp=retp, retsc=retsc),
+   function(rets) {
+     Box.test(rets, lag=10, type="Ljung")$statistic
+ })  ##  end sapply
> # Ljung-Box test statistics for aggregated SPY returns
> sapply(list(minutely=retp, hourly=retsh, daily=retd),
+   function(rets) {
+     Box.test(rets, lag=10, type="Ljung")$statistic
+ })  ##  end sapply
```

The level of the autocorrelations depends on the sampling frequency, with higher frequency returns having more significant negative autocorrelations.

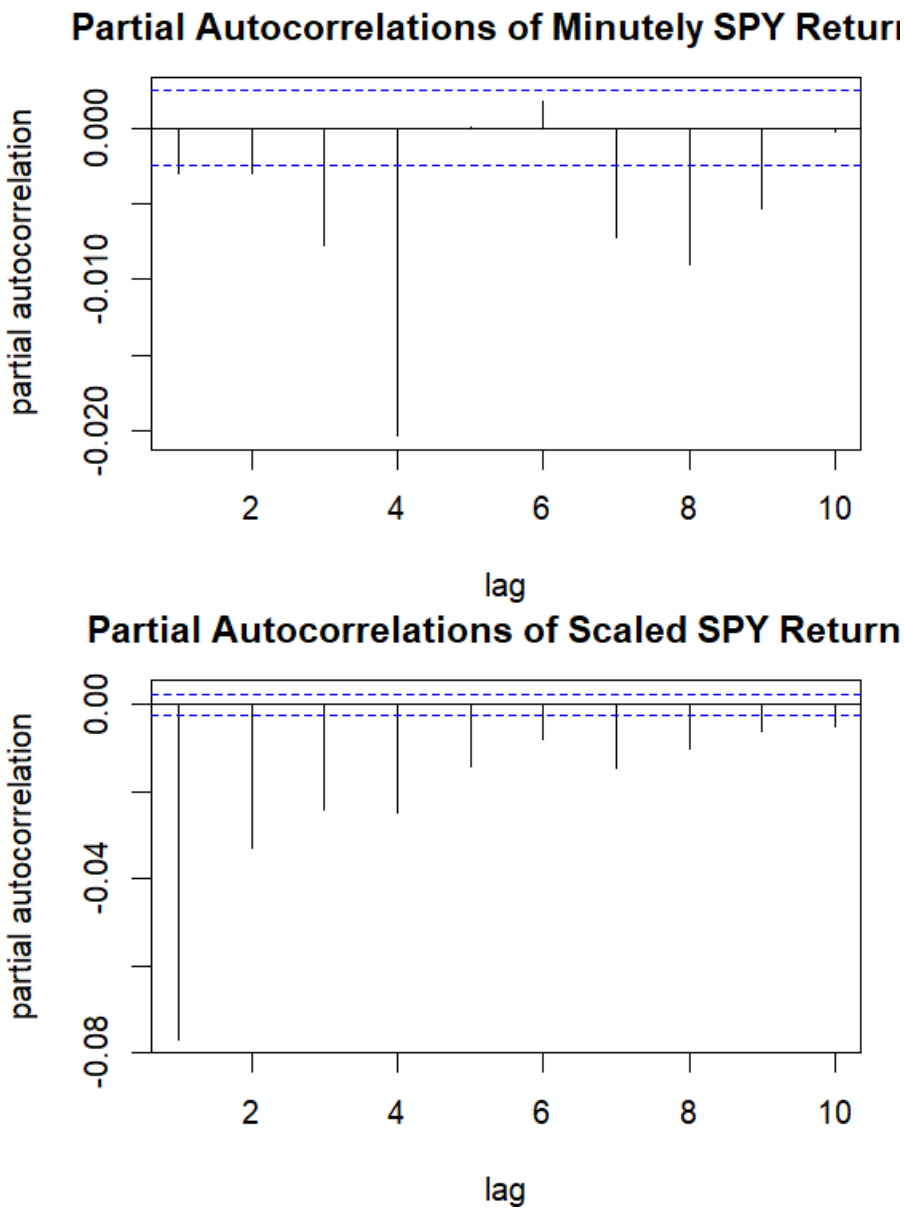
As the returns are aggregated to a lower periodicity, they become less autocorrelated, with daily returns having almost insignificant autocorrelations.

Partial Autocorrelations of High Frequency Returns

High frequency minutely *SPY* returns have statistically significant negative autocorrelations.

SPY returns *scaled* by the trading volumes have even more significant negative autocorrelations.

```
> # Set plot parameters
> x11(width=6, height=8)
> par(mar=c(4, 4, 2, 1), oma=c(0, 0, 0, 0))
> layout(matrix(c(1, 2), ncol=1), widths=c(6, 6), heights=c(4, 4))
> # Plot the partial autocorrelations of minutely SPY returns
> pacfl <- pacf(as.numeric(retp), lag=10,
+       xlab="lag", ylab="partial autocorrelation", main="")
> title("Partial Autocorrelations of Minutely SPY Returns", line=1)
> # Plot the partial autocorrelations of scaled SPY returns
> pacfs <- pacf(as.numeric(retsc), lag=10,
+       xlab="lag", ylab="partial autocorrelation", main="")
> title("Partial Autocorrelations of Scaled SPY Returns", line=1)
> # Calculate the sums of partial autocorrelations
> sum(pacfl$acf)
> sum(pacfs$acf)
```



Market Liquidity, Trading Volume and Volatility

Market illiquidity is defined as the market price impact resulting from supply-demand imbalance.

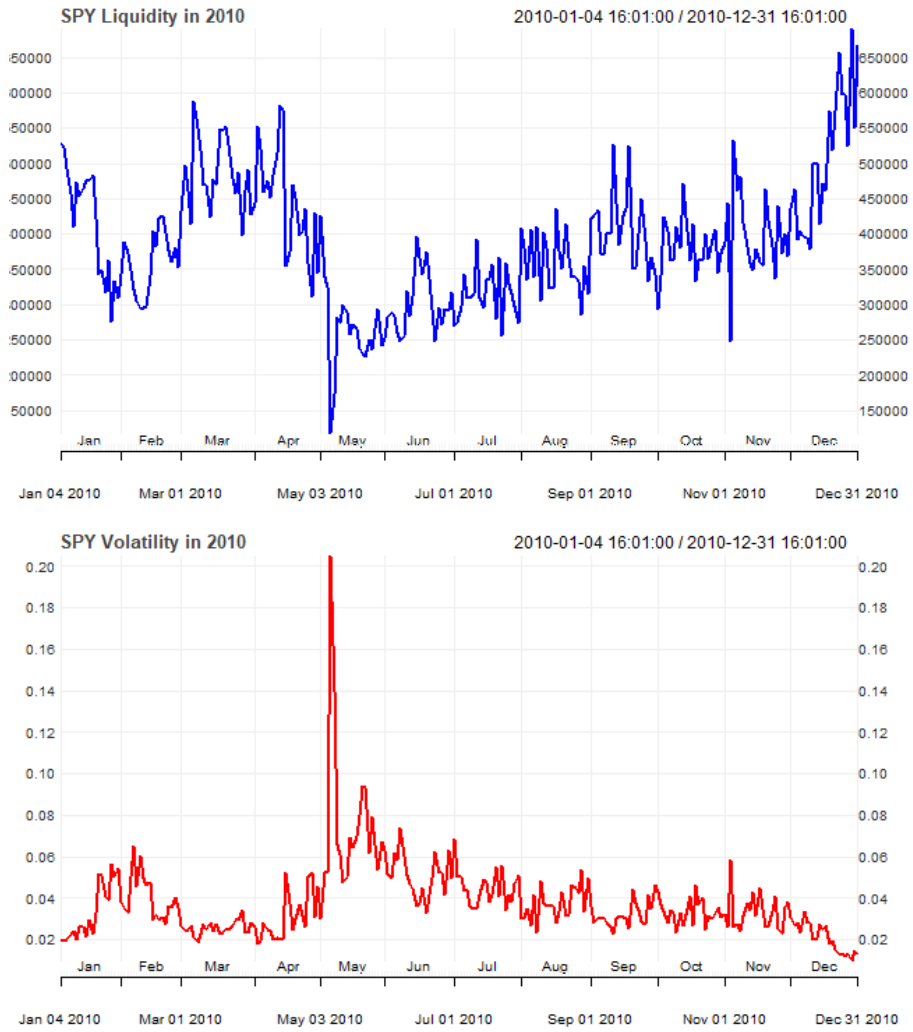
Market liquidity $\mathcal{L}$ is proportional to the square root of the *trading volume* υ divided by the price volatility σ:

$$\mathcal{L} \sim \frac{\sqrt{\upsilon}}{\sigma}$$

Market illiquidity spiked during the May 6, 2010 *flash crash*.

Research suggests that market crashes are caused by declining market liquidity:
Donier et al., Why Do Markets Crash?

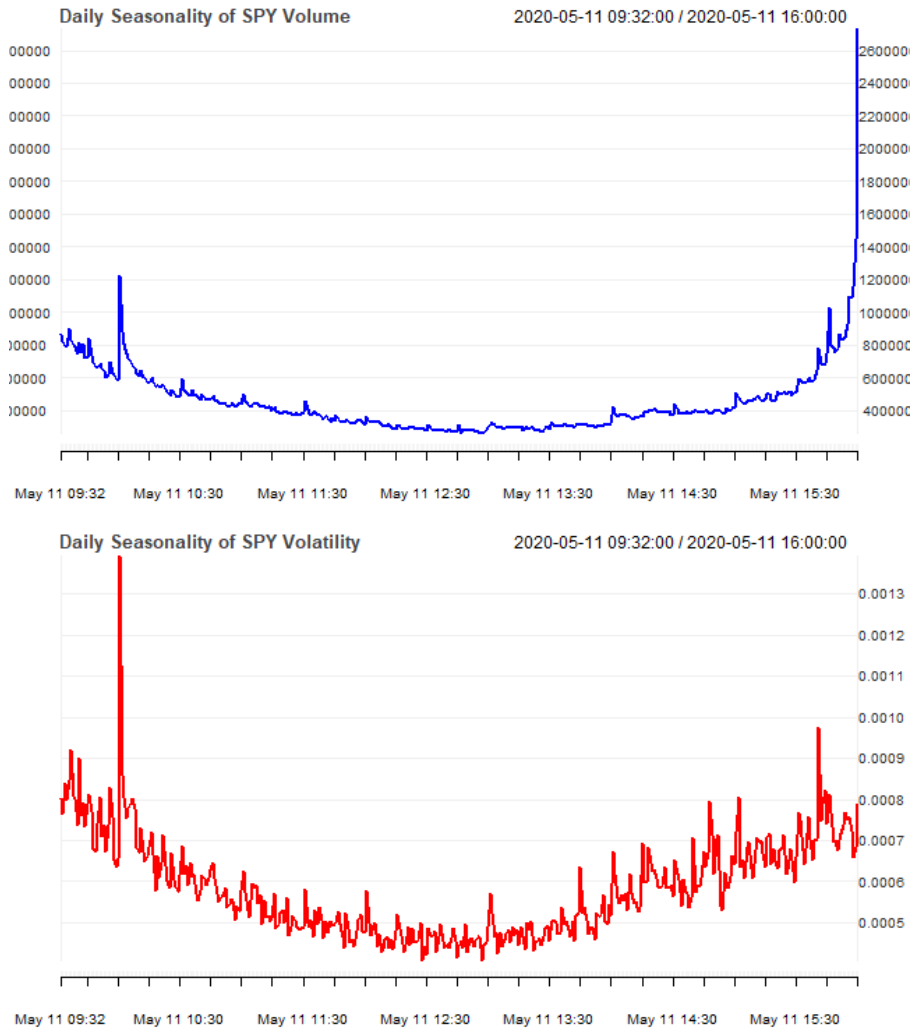
```
> # Calculate market illiquidity
> liquidv <- sqrt(volumd)/volatd
> # Plot market illiquidity
> x11(width=6, height=7) ; par(mfrow=c(2, 1))
> themev <- chart_theme()
> themev$col$line.col <- c("blue")
> chart_Series(liquidv["2010"], theme=themev,
+   name="SPY Liquidity in 2010", plot=FALSE)
> themev$col$line.col <- c("red")
> chart_Series(volatd["2010"],
+   theme=themev, name="SPY Volatility in 2010")
```



Intraday Seasonality of Volume and Volatility

The volatility and trading volumes are typically higher at the beginning and end of the trading sessions.

```
> # Calculate intraday time index with hours and minutes
> datev <- format(zoo::index(retp), "%H:%M")
> # Aggregate the mean volume
> volumagg <- tapply(X=volumv, INDEX=datev, FUN=mean)
> volumagg <- drop(volumagg)
> # Aggregate the mean volatility
> volagg <- tapply(X=retp^2, INDEX=datev, FUN=mean)
> volagg <- sqrt(drop(volagg))
> # Coerce to xts
> datev <- as.POSIXct(paste(Sys.Date(), names(volumagg)))
> volumagg <- xts::xts(volumagg, datev)
> volagg <- xts::xts(volagg, datev)
> # Plot seasonality of volume and volatility
> x11(width=6, height=7) ; par(mfrow=c(2, 1))
> themev <- chart_theme()
> themev$col$line.col <- c("blue")
> chart_Series(volumagg[c(-1, -NROW(volumagg))], theme=themev,
+   name="Intraday Seasonality of SPY Volume", plot=FALSE)
> themev$col$line.col <- c("red")
> chart_Series(volagg[c(-1, -NROW(volagg))], theme=themev,
+   name="Intraday Seasonality of SPY Volatility")
```

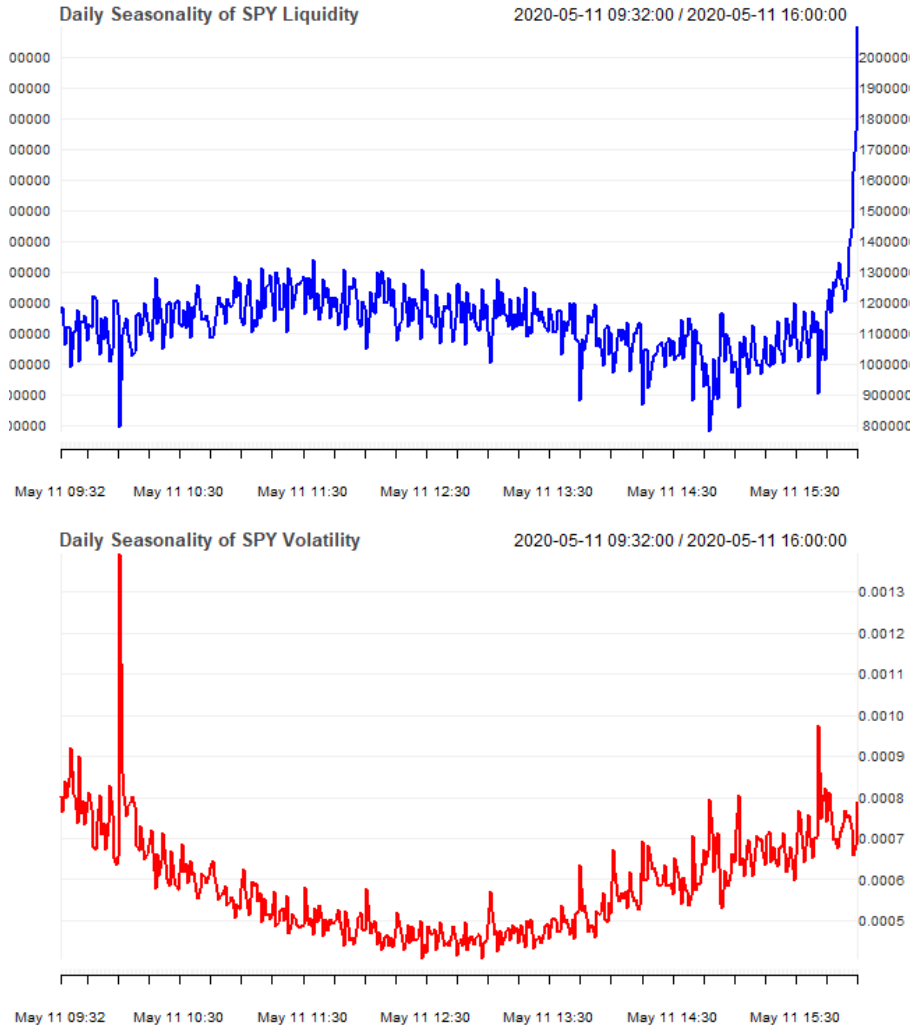


Intraday Seasonality of Liquidity and Volatility

Market liquidity is typically the highest at the end of the trading session, and the lowest at the beginning.

The end of day spike in trading volumes and liquidity is driven by computer-driven investors liquidating their positions.

```
> # Calculate market liquidity
> liquidv <- sqrt(volumagg)/volagg
> # Plot intraday seasonality of market liquidity
> x11(width=6, height=7) ; par(mfrow=c(2, 1))
> themev <- chart_theme()
> themev$col$line.col <- c("blue")
> chart_Series(liquidv[c(-1, -NROW(liquidv))], theme=themev,
+   name="Intraday Seasonality of SPY Liquidity", plot=FALSE)
> themev$col$line.col <- c("red")
> chart_Series(volagg[c(-1, -NROW(volagg))], theme=themev,
+   name="Intraday Seasonality of SPY Volatility")
```



Flattening a List of Vectors to a Matrix Using do.call()

A list of vectors can be flattened into a matrix using the functions do.call() and either rbind() or cbind().

If the list contains vectors of different lengths, then R applies the recycling rule.

If the list contains a NULL element, that element is skipped.

```
> # Create list of vectors
> listv <- lapply(1:3, function(x) sample(6))
> # Bind list elements into matrix - doesn't work
> rbind(listv)
> # Bind list elements into matrix - tedious
> rbind(listv[[1]], listv[[2]], listv[[3]])
> # Bind list elements into matrix - works!
> do.call(rbind, listv)
> # Create numeric list
> listv <- list(1, 2, 3, 4)
> do.call(rbind, listv)  ##  Returns single column matrix
> do.call(cbind, listv)  ##  Returns single row matrix
> # Recycling rule applied
> do.call(cbind, list(1:2, 3:5))
> # NULL element is skipped
> do.call(cbind, list(1, NULL, 3, 4))
> # NA element isn't skipped
> do.call(cbind, list(1, NA, 3, 4))
```

Efficient Binding of Lists Into Matrices

A list of vectors can be flattened into a matrix using the functions `do.call()` and either `rbind()` or `cbind()`.

But for large vectors this procedure can be very slow, and often causes an out of memory error.

The function `do_call_rbind()` efficiently combines a list of vectors into a matrix.

`do_call_rbind()` produces the same result as `do.call(rbind, list_var)`, but using recursion.

`do_call_rbind()` calls lapply in a loop, each time binding neighboring list elements and dividing the length of the list by half.

`do_call_rbind()` is the same function as `do.call.rbind()` from package *qmao*:
https://r-forge.r-project.org/R/?group_id=1113

```
> listv <- lapply(1:5, rnorm, n=10)
> matv <- do.call(rbind, listv)
> dim(matv)
> do_call_rbind <- function(listv) {
+   while (NROW(listv) > 1) {
+ # Index of odd list elements
+     odd_index <- seq(from=1, to=NROW(listv), by=2)
+ # Bind odd and even elements, and divide listv by half
+     listv <- lapply(odd_index, function(indeks) {
+ if (indeks==NROW(listv)) return(listv[[indeks]])
+ rbind(listv[[indeks]], listv[[indeks+1]])
+     })  ##  end lapply
+   }  ##  end while
+ # listv has only one element - return it
+   listv[[1]]
+ }  ##  end do_call_rbind
> all.equal(matv, do_call_rbind(listv))
```

Filtering Data Frames Using subset()

Filtering means extracting rows from a *data frame* that satisfy a logical condition.

Data frames can be filtered using Boolean vectors and brackets "[]" operators.

The function subset() filters *data frames*, by applying logical conditions to its columns, using the column names.

subset() provides a succinct notation and discards NA values, but it's slightly slower than using Boolean vectors and brackets "[]" operators.

```
> airquality[(airquality$Solar.R > 320 & !is.na(airquality$Solar.R)),
> subset(x=airquality, subset=(Solar.R > 320))
> summary(microbenchmark(
+     subset=subset(x=airquality, subset=(Solar.R > 320)),
+     brackets=airquality[(airquality$Solar.R > 320 &
+             !is.na(airquality$Solar.R)), ],
+ times=10))[, c(1, 4, 5)]  ##  end microbenchmark summary
```

Splitting Data Frames Using factor Categorical Variables

The function split() divides an object into a list of objects, according to a factor (categorical variable).

The list's names attribute is equal to the factor levels.

```
> unique(iris$Species)  ##  Species has three distinct values
> # Split into separate data frames by hand
> setosa <- iris[iris$Species=="setosa", ]
> versi <- iris[iris$Species=="versicolor", ]
> virgin <- iris[iris$Species=="virginica", ]
> dim(setosa)
> head(setosa, 2)
> # Split iris into list based on Species
> irisplit <- split(iris, iris$Species)
> str(irisplit, max.confl=1)
> names(irisplit)
> dim(irisplit$setosa)
> head(irisplit$setosa, 2)
> all.equal(setosa, irisplit$setosa)
```

The *split-apply-combine* Procedure

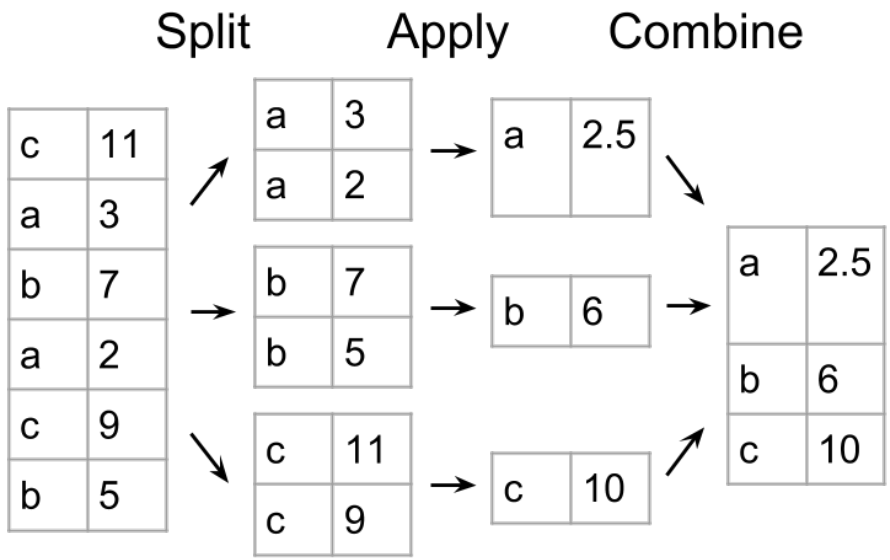
The *split-apply-combine* procedure consists of:

- dividing an object into a list, according to a factor (attribute).
- applying a function to each list element.
- combining the results.

The *split-apply-combine* procedure is also called the *map-reduce* procedure, or simply *data pivoting*, and it's similar to *pivot tables* in *Excel*.

Data pivoting can be performed *data frames*, by aggregating its columns based on categorical data stored in one of its columns.

You can read more about the *split-apply-combine* procedure in Hadley Wickham's paper:
http://www.jstatsoft.org/v40/i01/paper



Data Pivoting Example

Data pivoting can be performed through successive applications of functions `split()`, `apply()`, and `unlist()`.

A *data frame* can be *pivoted* either by first splitting it into a list of *data frames* and then aggregating, or by splitting just a single column and aggregating it.

The function `split()` divides an object into a list of objects, according to a `factor` (categorical variable).

The list's `names` attribute is equal to the `factor` levels.

The functional `aggregate()` *pivots* the columns of a *data frame*.

`aggregate()` can accept a `"formula"` argument with the column names, or it can accept `"x"` and `"by"` arguments with the columns.

`aggregate()` returns a *data frame* containing the names of the groups (`factor` confls).

```
> unique(mtcars$cyl)  ##  cyl has three unique values
> # Split mpg column based on number of cylinders
> split(mtcars$mpg, mtcars$cyl)
> # Split mtcars data frame based on number of cylinders
> carsplit <- split(mtcars, mtcars$cyl)
> str(carsplit, max.confl=1)
> names(carsplit)
> # Aggregate the mean mpg over split mtcars data frame
> sapply(carsplit, function(x) mean(x$mpg))
> # Or: split mpg column and aggregate the mean
> sapply(split(mtcars$mpg, mtcars$cyl), mean)
> # Same but using with()
> with(mtcars, sapply(split(mpg, cyl), mean))
> # Or: aggregate() using formula syntax
> aggregate(x=(mpg ~ cyl), data=mtcars, FUN=mean)
> # Or: aggregate() using data frame syntax
> aggregate(x=mtcars$mpg, by=list(cyl=mtcars$cyl), FUN=mean)
> # Or: using name for mpg
> aggregate(x=list(mpg=mtcars$mpg), by=list(cyl=mtcars$cyl), FUN=mea
> # Aggregate() all columns
> aggregate(x=mtcars, by=list(cyl=mtcars$cyl), FUN=mean)
> # Aggregate multiple columns using formula syntax
> aggregate(x=(cbind(mpg, hp) ~ cyl), data=mtcars, FUN=mean)
```

The tapply() Functional

The functional tapply() is a specialized version of the apply() functional, that applies a function to elements of a *jagged array*.

A *jagged array* is a list consisting of vectors or matrices of different lengths.

tapply() accepts a vector of values "X", a factor "INDEX", and a function "FUN".

tapply() first groups the elements of "X" according to the factor "INDEX", transforming it into a *jagged array*, and then applies "FUN" to each element of the *jagged array*.

tapply() applies a function to sub-vectors aggregated using a factor, and performs *data pivoting* in a single function call.

The by() function is a wrapper for tapply().

The with() function evaluates an expression in an environment constructed from the data.

```
> # Mean mpg for each cylinder group
> tapply(X=mtcars$mpg, INDEX=mtcars$cyl, FUN=mean)
> # using with() environment
> with(mtcars, tapply(X=mpg, INDEX=cyl, FUN=mean))
> # Function sapply() instead of tapply()
> with(mtcars, sapply(sort(unique(cyl)), function(x) {
+   structure(mean(mpg[x==cyl]), names=x)
+ }))  ##  end with
> # Function by() instead of tapply()
> with(mtcars, by(data=mpg, INDICES=cyl, FUN=mean))
```

Data Pivoting Returning a Matrix

Sometimes *data pivoting* returns a list of vectors.

A list of vectors can be flattened into a matrix using the functions `do.call()` and either `rbind()` or `cbind()`.

The function `do.call()` executes a function call using a function name and a list of arguments.

`do.call()` passes the list elements individually, instead of passing the whole list as one argument: `do.call(fun, list)= fun(list[[1]], list[[2]], ...)`

```
> # Get several mpg stats for each cylinder group
> cardata <- sapply(carsplit, function(x) {
+   c(mean=mean(x$mpg), max=max(x$mpg), min=min(x$mpg))
+ }  ##  end anonymous function
+ )  ##  end sapply
> cardata  ##  sapply() produces a matrix
> # Now same using lapply
> cardata <- lapply(carsplit, function(x) {
+   c(mean=mean(x$mpg), max=max(x$mpg), min=min(x$mpg))
+ }  ##  end anonymous function
+ )  ##  end sapply
> is.list(cardata)  ##  lapply produces a list
> # do.call flattens list into a matrix
> do.call(cbind, cardata)
```

Data Pivoting of Panel Data

The *data frame* `panel_data` contains fundamental financial data for *S&P500* stocks.

The `Industry` column has 22 unique elements, while the `Sector` column has 10 unique elements.

Each `Industry` belongs to a single `Sector`, but each `Sector` may have several `Industries` that belong to it.

The functional `aggregate()` allows aggregating over the `Industry` column, by performing *data pivoting*.

```
> # Download CRSPpanel.txt from the NYU share drive
> # Read the file using read.table() with header and sep arguments
> paneld <- read.table(file="/Users/jerzy/Develop/lecture_slides/dat
+                      header=TRUE, sep="\t")
> paneld[1:5, 1:5]
> attach(paneld)
> # Split paneld based on Industry column
> panelds <- split(paneld, Industry)
> # Number of companies in each Industry
> sapply(panelds, NROW)
> # Number of Sectors that each Industry belongs to
> sapply(panelds, function(x) {
+   NROW(unique(x$Sector))
+ })  ##  end sapply
> # Or
> aggregate(x=(Sector ~ Industry),
+   data=paneld, FUN=function(x) NROW(unique(x)))
> # Industries and the Sector to which they belong
> aggregate(x=(Sector ~ Industry), data=paneld, FUN=unique)
> # Or
> aggregate(x=Sector, by=list(Industry), FUN=unique)
> # Or
> sapply(unique(Industry), function(x) {
+   Sector[match(x, Industry)]
+ })  ##  end sapply
```

Data Pivoting Returning a *Jagged Array*

A *jagged array* is a list consisting of vectors or matrices of different lengths.

The functional `aggregate()` returns a *data frame*, so it's output must be coerced if the *data pivoting* attempts to return a *jagged array*.

The functional `tapply()` returns an array, so it's output must be coerced if the *data pivoting* attempts to return a *jagged array*.

`tapply()` accepts a vector of values `"X"`, a factor `"INDEX"`, and a function `"FUN"`.

`tapply()` first groups the elements of `"X"` according to the factor `"INDEX"`, transforming it into a *jagged array*, and then applies `"FUN"` to each element of the *jagged array*.

`tapply()` applies a function to sub-vectors aggregated using a factor, and performs *data pivoting* in a single function call.

```
> # Split paneld based on Sector column
> panelds <- split(paneld, Sector)
> # Number of companies in each Sector
> sapply(panelds, NROW)
> # Industries belonging to each Sector (jagged array)
> secind <- sapply(panelds, function(x) unique(x$Industry))
> # Or use aggregate() (returns a data frame)
> secind2 <- aggregate(x=(Industry ~ Sector),
+   data=paneld, FUN=function(x) unique(x))
> # Or use aggregate() with "by" argument
> secind2 <- aggregate(x=Industry, by=list(Sector),
+     FUN=function(x) as.vector(unique(x)))
> # Coerce secind2 into a jagged array
> namev <- secind2[, 1]
> secind2 <- secind2[, 2]
> names(secind2) <- namev
> all.equal(secind2, secind)
> # Or use tapply() (returns an array)
> secind2 <- tapply(X=Industry, INDEX=Sector, FUN=unique)
> # Coerce secind2 into a jagged array
> secind2 <- drop(as.matrix(secind2))
> all.equal(secind2, secind)
```

Data Pivoting Over Multiple Columns

Data pivoting over multiple columns can be performed by splitting the *data frame* and then performing an sapply() loop using an anonymous function.

Splitting the *data frame* allows aggregations over multiple columns.

An anonymous function allows applying different aggregations on the same column.

```
> # Average ROE in each Industry
> sapply(split(ROE, Industry), mean)
> # Average, min, and max ROE in each Industry
> t(sapply(split(ROE, Industry), FUN=function(x)
+   c(mean=mean(x), max=max(x), min=min(x))))
> # Split paneld based on Industry column
> panelds <- split(paneld, Industry)
> # Average ROE and EPS in each Industry
> t(sapply(panelds, FUN=function(x)
+   c(mean_roe=mean(x$ROE),
+     mean_eps=mean(x$EPS.EXCLUDE.EI))))
> # Or: split paneld based on Industry column
> panelds <- split(paneld[, c("ROE", "EPS.EXCLUDE.EI")],
+   paneld$Industry)
> # Average ROE and EPS in each Industry
> t(sapply(panelds, FUN=function(x) sapply(x, mean)))
> # Average ROE and EPS using aggregate()
> aggregate(x=paneld[, c("ROE", "EPS.EXCLUDE.EI")],
+   by=list(paneld$Industry), FUN=mean)
```

Date Objects

R has a Date class for date objects (but without time).

The function as.Date() parses character strings and coerces numeric objects into Date objects.

R stores Date objects as the number of days since the *epoch* (January 1, 1970).

The function difftime() calculates the difference between Date objects, and returns a time interval object of class difftime.

The "+" and "-" arithmetic operators and the "<" and ">" logical comparison operators are overloaded to allow these operations directly on Date objects.

numeric *year-fraction* dates can be coerced to Date objects using the functions attributes() and structure().

```
> Sys.Date()  ##  Get today's date
> as.Date(1e3)  ##  Coerce numeric into date object
> datetime <- as.Date("2014-07-14")  ##  "%Y-%m-%d" or "%Y/%m/%d"
> datetime
> class(datetime)  ##  Date object
> as.Date("07-14-2014", "%m-%d-%Y")  ##  Specify format
> datetime + 20  ##  Add 20 days
> # Extract internal representation to integer
> as.numeric(datetime)
> datep <- as.Date("07/14/2013", "%m/%d/%Y")
> datep
> # Difference between dates
> difftime(datetime, datep, units="weeks")
> weekdays(datetime)  ##  Get day of the week
> # Coerce numeric into date-times
> datetime <- 0
> attributes(datetime) <- list(class="Date")
> datetime  ##  "Date" object
> structure(0, class="Date")  ##  "Date" object
> structure(10000.25, class="Date")
```

POSIXct Date-time Objects

The POSIXct class in R represents *date-time* objects, that can store both the date and time.

The *clock time* is the time (number of hours, minutes and seconds) in the local *time zone*.

The *moment of time* is the *clock time* in the UTC *time zone*.

POSIXct objects are stored as the number of seconds that have elapsed since the *epoch* (January 1, 1970) in the UTC *time zone*.

POSIXct objects are stored as the *moment of time*, but are printed out as the *clock time* in the local *time zone*.

A *clock time* together with a *time zone* uniquely specifies a *moment of time*.

The function as.POSIXct() can parse a character string (representing the *clock time*) and a *time zone* into a POSIXct object.

POSIX is an acronym for "Portable Operating System Interface".

```
> datetime <- Sys.time()  ##  Get today's date and time
> datetime
> class(datetime)  ##  POSIXct object
> # POSIXct stored as integer moment of time
> as.numeric(datetime)
> # Parse character string "%Y-%m-%d %H:%M:%S" to POSIXct object
> datetime <- as.POSIXct("2014-07-14 13:30:10")
> # Different time zones can have same clock time
> as.POSIXct("2014-07-14 13:30:10", tz="America/New_York")
> as.POSIXct("2014-07-14 13:30:10", tz="UTC")
> # Format argument allows parsing different date-time string format
> as.POSIXct("07/14/2014 13:30:10", format="%m/%d/%Y %H:%M:%S",
+      tz="America/New_York")
```

Operations on POSIXct Objects

The "+" and "-" arithmetic operators are overloaded to allow addition and subtraction operations on POSIXct objects.

The "<" and ">" logical comparison operators are also overloaded to allow direct comparisons between POSIXct objects.

Operations on POSIXct objects are equivalent to the same operations on the internal integer representation of POSIXct (number of seconds since the *epoch*).

Subtracting POSIXct objects creates a time interval object of class difftime.

The method seq.POSIXt creates a vector of POSIXct *date-times*.

```
> # Same moment of time corresponds to different clock times
> timeny <- as.POSIXct("2014-07-14 13:30:10", tz="America/New_York")
> timeldn <- as.POSIXct("2014-07-14 13:30:10", tz="UTC")
> # Add five hours to POSIXct
> timeny + 5*60*60
> # Subtract POSIXct
> timeny - timeldn
> class(timeny - timeldn)
> # Compare POSIXct
> timeny > timeldn
> # Create vector of POSIXct times during trading hours
> timev <- seq(
+   from=as.POSIXct("2014-07-14 09:30:00", tz="America/New_York"),
+   to=as.POSIXct("2014-07-14 16:00:00", tz="America/New_York"),
+   by="10 min")
> head(timev, 3)
> tail(timev, 3)
```

Moment of Time and Clock Time

`as.POSIXct()` can also coerce `integer` objects into `POSIXct`, given an `origin` in time.

The same *moment of time* corresponds to different *clock times* in different *time zones*.

The same *clock times* in different *time zones* correspond to different *moments of time*.

```
> # POSIXct is stored as integer moment of time
> datetimen <- as.numeric(datetime)
> # Same moment of time corresponds to different clock times
> as.POSIXct(datetimen, origin="1970-01-01", tz="America/New_York")
> as.POSIXct(datetimen, origin="1970-01-01", tz="UTC")
> # Same clock time corresponds to different moments of time
> as.POSIXct("2014-07-14 13:30:10", tz="America/New_York") -
+   as.POSIXct("2014-07-14 13:30:10", tz="UTC")
> # Add 20 seconds to POSIXct
> datetime + 20
```

Methods for Manipulating POSIXct Objects

The generic function `format()` formats R objects for printing and display.

The method `format.POSIXct()` parses `POSIXct` objects into a `character` string representing the *clock time* in a given *time zone*.

The method `as.POSIXct.Date()` parses `Date` objects into `POSIXct`, and assigns to them the *moment of time* corresponding to midnight `UTC`.

`POSIX` is an acronym for "Portable Operating System Interface".

```
> datetime  ##  POSIXct date and time
> # Parse POSIXct to string representing the clock time
> format(datetime)
> class(format(datetime))  ##  Character string
> # Get clock times in different time zones
> format(datetime, tz="America/New_York")
> format(datetime, tz="UTC")
> # Format with custom format strings
> format(datetime, "%m/%Y")
> format(datetime, "%m-%d-%Y %H hours")
> # Trunc to hour
> format(datetime, "%m-%d-%Y %H:00:00")
> # Date converted to midnight UTC moment of time
> as.POSIXct(Sys.Date())
> as.POSIXct(as.numeric(as.POSIXct(Sys.Date())),
+       origin="1970-01-01",
+       tz="UTC")
```

POSIXlt Date-time Objects

The POSIXlt class in R represents *date-time* objects, that are stored internally as a list.

The function as.POSIXlt() can parse a character string (representing the *clock time*) and a *time zone* into a POSIXlt object.

The method format.POSIXlt() parses POSIXlt objects into a character string representing the *clock time* in a given *time zone*.

The function as.POSIXlt() can also parse a POSIXct object into a POSIXlt object, and as.POSIXct() can perform the reverse.

Adding a number to POSIXlt causes implicit coercion to POSIXct.

POSIXct and POSIXlt are two derived classes from the POSIXt class.

The methods round.POSIXt() and trunc.POSIXt() round and truncate POSIXt objects, and return POSIXlt objects.

```
> # Parse character string "%Y-%m-%d %H:%M:%S" to POSIXlt object
> datetime <- as.POSIXlt("2014-07-14 18:30:10")
> datetime
> class(datetime)  ##  POSIXlt object
> as.POSIXct(datetime)  ##  Coerce to POSIXct object
> # Extract internal list representation to vector
> unlist(datetime)
> datetime + 20  ##  Add 20 seconds
> class(datetime + 20)  ##  Implicit coercion to POSIXct
> trunc(datetime, units="hours")  ##  Truncate to closest hour
> trunc(datetime, units="days")  ##  Truncate to closest day
> methods(trunc)  ##  Trunc methods
> trunc.POSIXt
```

Time Zones and Date-time Conversion

date-time objects require a *time zone* to be uniquely specified.

`UTC` stands for "Universal Time Coordinated", and is synonymous with `GMT`, but doesn't change with Daylight Saving Time.

`EST` stands for "Eastern Standard Time", and is `UTC - 5` hours.

`EDT` stands for "Eastern Daylight Time", and is `UTC - 4` hours.

The function `Sys.setenv()` can be used to set the default *time zone*, but the environment variable `"TZ"` must be capitalized.

```
> # Set time-zone to UTC
> Sys.setenv(TZ="UTC")
> Sys.timezone()  ##  Get time-zone
> Sys.time()  ##  Today's date and time
> # Set time-zone back to New York
> Sys.setenv(TZ="America/New_York")
> Sys.time()  ##  Today's date and time
> # Standard Time in effect
> as.POSIXct("2013-03-09 11:00:00", tz="America/New_York")
> # Daylight Savings Time in effect
> as.POSIXct("2013-03-10 11:00:00", tz="America/New_York")
> datetime <- Sys.time()  ##  Today's date and time
> # Convert to character in different TZ
> format(datetime, tz="America/New_York")
> format(datetime, tz="UTC")
> # Parse back to POSIXct
> as.POSIXct(format(datetime, tz="America/New_York"))
> # Difference between New_York time and UTC
> as.POSIXct(format(Sys.time(), tz="UTC")) -
+   as.POSIXct(format(Sys.time(), tz="America/New_York"))
```

Manipulating Date-time Objects Using *lubridate*

The package *lubridate* contains functions for manipulating `POSIXct` date-time objects.

The `ymd()`, `dmy()`, etc. functions parse `character` and `numeric` *year-fraction* dates into `POSIXct` objects.

The `mday()`, `month()`, `year()`, etc. accessor functions extract date-time components.

The function `decimal_date()` converts `POSIXct` objects into `numeric` *year-fraction* dates.

The function `date_decimal()` converts `numeric` *year-fraction* dates into `POSIXct` objects.

```
> library(lubridate)  ##  Load lubridate
> # Parse strings into date-times
> as.POSIXct("07-14-2014", format="%m-%d-%Y", tz="America/New_York")
> datetime <- lubridate::mdy("07-14-2014", tz="America/New_York")
> datetime
> class(datetime)  ##  POSIXct object
> lubridate::dmy("14.07.2014", tz="America/New_York")
>
> # Parse numeric into date-times
> as.POSIXct(as.character(14072014), format="%d%m%Y",
+                    tz="America/New_York")
> lubridate::dmy(14072014, tz="America/New_York")
>
> # Parse decimal to date-times
> lubridate::decimal_date(datetime)
> lubridate::date_decimal(2014.25, tz="America/New_York")
> date_decimal(decimal_date(datetime), tz="America/New_York")
```

Time Zones Using *lubridate*

The package *lubridate* simplifies *time zone* calculations.

The package *lubridate* uses the *UTC time zone* as default.

The function `with_tz()` creates a date-time object with the same moment of time in a different *time zone*.

The function `force_tz()` creates a date-time object with the same clock time in a different *time zone*.

```
> datetime <- lubridate::ymd_hms(20140714142010,
+                tz="America/New_York")
> datetime
> # Get same moment of time in "UTC" time zone
> lubridate::with_tz(datetime, "UTC")
> as.POSIXct(format(datetime, tz="UTC"), tz="UTC")
> # Get same clock time in "UTC" time zone
> lubridate::force_tz(datetime, "UTC")
> as.POSIXct(format(datetime, tz="America/New_York"),
+      tz="UTC")
> # Same moment of time
> datetime - with_tz(datetime, "UTC")
> # Different moments of time
> datetime - force_tz(datetime, "UTC")
```

lubridate Time Span Objects

lubridate has two time span classes: `durations` and `periods`.

`durations` specify exact time spans, such as numbers of seconds, hours, days, etc.

The functions `ddays()`, `dyears()`, etc. return `duration` objects.

`periods` specify relative time spans that don't have a fixed length, such as months, years, etc.

`periods` account for variable days in the months, for Daylight Savings Time, and for leap years.

The functions `days()`, `months()`, `years()`, etc. return `period` objects.

```
> # Daylight Savings Time handling periods vs durations
> datetime <- as.POSIXct("2013-03-09 11:00:00", tz="America/New_York
> datetime
> datetime + lubridate::ddays(1)  ##  Add duration
> datetime + lubridate::days(1)  ##  Add period
>
> leap_year(2012)  ##  Leap year
> datetime <- lubridate::dmy(01012012, tz="America/New_York")
> datetime
> datetime + lubridate::dyears(1)  ##  Add duration
> datetime + lubridate::years(1)  ##  Add period
```

Adding Time Spans to Date-time Objects

`periods` allow calculating future dates with the same day of the month, or month of the year.

```
> datetime <- lubridate::ymd_hms(20140714142010, tz="America/New_Yo
> datetime
> # Add periods to a date-time
> c(datetime + lubridate::seconds(1), datetime + lubridate::minutes
+   datetime + lubridate::days(1), datetime + period(months=1))
>
> # Create vectors of dates
> datetime <- lubridate::ymd(20140714, tz="America/New_York")
> datetime + 0:2 * period(months=1)  ##  Monthly dates
> datetime + period(months=0:2)
> datetime + 0:2 * period(months=2)  ##  bi-monthly dates
> datetime + seq(0, 5, by=2) * period(months=1)
> seq(datetime, length=3, by="2 months")
```

End-of-month Dates

Adding monthly `periods` can create invalid dates.

The operators `%m+%` and `%m-%` add or subtract monthly `periods` to account for the varible number of days per month.

This allows creating vectors of end-of-month dates.

```
> # Adding monthly periods can create invalid dates
> datetime <- lubridate::ymd(20120131, tz="America/New_York")
> datetime + 0:2 * period(months=1)
> datetime + period(months=1)
> datetime + period(months=2)
> # Create vector of end-of-month dates
> datetime %m-% months(13:1)
```

Package *RQuantLib* Calendar Functions

The package *RQuantLib* is an interface to the *QuantLib* open source C/C++ library for quantitative finance, mostly designed for pricing fixed-income instruments and options.

The *QuantLib* library also contains calendar functions for determining holidays and business days in many different jurisdictions.

```
> library(RQuantLib)  ##  Load RQuantLib
>
> # Create daily date series of class "Date"
> datev <- Sys.Date() + -5:2
> datev
[1] "2025-11-26" "2025-11-27" "2025-11-28" "2025-11-29" "2025-11-30"
[6] "2025-12-01" "2025-12-02" "2025-12-03"
>
> # Create Boolean vector of business days
> # Use RQuantLib calendar
> isbusday <- RQuantLib::isBusinessDay(
+   calendar="UnitedStates/GovernmentBond", datev)
>
> # Create daily series of business days
> datev[isbusday]
[1] "2025-11-26" "2025-11-28" "2025-12-01" "2025-12-02" "2025-12-03"
```

Review of Date-time Classes in R

The `Date` class from the `base` package is suitable for *daily* time series.

The `POSIXct` class from the `base` package is suitable for *intra-day* time series.

The `yearmon` and `yearqtr` classes from the *zoo* package are suitable for *quarterly* and *monthly* time series.

```
> datetime <- Sys.Date()  ##  Create date series of class "Date"
> datev <- datetime + 0:365  ##  Daily series over one year
> head(datev, 4)  ##  Print first few dates
[1] "2025-12-01" "2025-12-02" "2025-12-03" "2025-12-04"
> format(head(datev, 4), "%m/%d/%Y")  ##  Print first few dates
[1] "12/01/2025" "12/02/2025" "12/03/2025" "12/04/2025"
> # Create daily date-time series of class "POSIXct"
> datev <- seq(Sys.time(), by="days", length.out=365)
> head(datev, 4)  ##  Print first few dates
[1] "2025-12-01 12:57:39 EST" "2025-12-02 12:57:39 EST"
[3] "2025-12-03 12:57:39 EST" "2025-12-04 12:57:39 EST"
> format(head(datev, 4), "%m/%d/%Y %H:%M:%S")  ##  Print first few
[1] "12/01/2025 12:57:39" "12/02/2025 12:57:39" "12/03/2025 12:57:39
[4] "12/04/2025 12:57:39"
> # Create series of monthly dates of class "zoo"
> monthv <- yearmon(2010+0:36/12)
> head(monthv, 4)  ##  Print first few dates
[1] "Jan 2010" "Feb 2010" "Mar 2010" "Apr 2010"
> # Create series of quarterly dates of class "zoo"
> qrtv <- yearqtr(2010+0:16/4)
> head(qrtv, 4)  ##  Print first few dates
[1] "2010 Q1" "2010 Q2" "2010 Q3" "2010 Q4"
> # Parse quarterly "zoo" dates to POSIXct
> Sys.setenv(TZ="UTC")
> as.POSIXct(head(qrtv, 4))
[1] "2010-01-01 UTC" "2010-04-01 UTC" "2010-07-01 UTC" "2010-10-01 U
```

Time Series Objects of Class *ts*

Time series are data objects that contain a *date-time* index and data associated with it.

The native time series class in `R` is *ts*.

ts time series are *regular*, i.e. they can only have an equally spaced *date-time* index.

ts time series have a `numeric` *date-time* index, usually encoded as a *year-fraction*, or some other unit, like number of months, etc.

For example the date `"2015-03-31"` can be encoded as a *year-fraction* equal to `2015.244`.

The *stats* base package contains functions for manipulating time series objects of class *ts*.

The function `ts()` creates a *ts* time series from a `numeric` vector or matrix, and from the associated *date-time* information (the number of data per time unit: year, month, etc.).

The `frequency` argument is the number of observations per unit of time.

For example, if the *date-time* index is encoded as a *year-fraction*, then `frequency=12` means 12 monthly data points per year.

```
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> # Create daily time series ending today
> startd <- decimal_date(Sys.Date()-6)
> endd <- decimal_date(Sys.Date())
> # Create vector of geometric Brownian motion
> datav <- exp(cumsum(rnorm(6)/100))
> tstep <- NROW(datav)/(endd-startd)
> timeser <- ts(data=datav, start=startd, frequency=tstep)
> timeser  ##  Display time series
> # Display index dates
> as.Date(date_decimal(zoo::coredata(time(timeser))))
> # bi-monthly geometric Brownian motion starting mid-1990
> timeser <- ts(data=exp(cumsum(rnorm(96)/100)),
+         frequency=6, start=1990.5)
```

Manipulating *ts* Time Series

ts time series don't store their *date-time* indices, and instead store only a `"tsp"` attribute that specifies the index `start` and `end` dates and its `frequency`.

The *date-time* index is calculated as needed from the `"tsp"` attribute.

The function `time()` extracts the *date-time* index of a *ts* time series object.

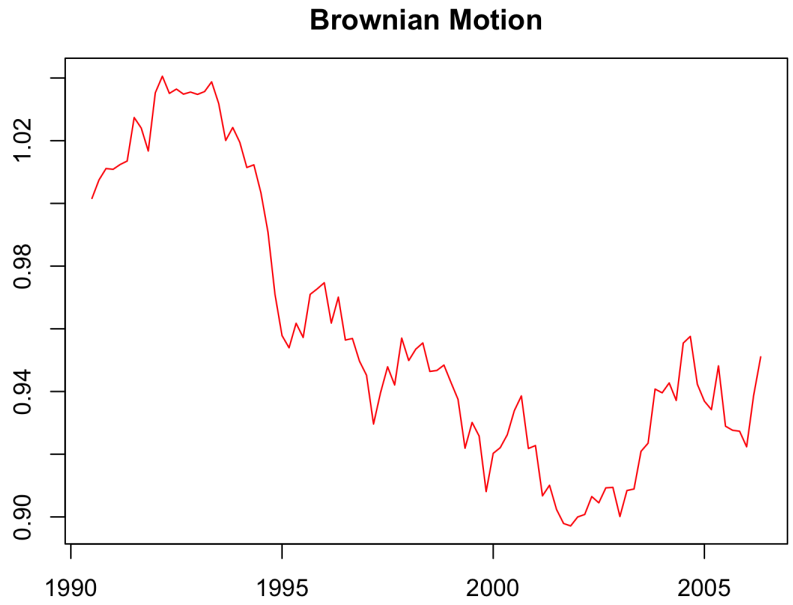
The function `window()` subsets the a *ts* time series object.

```
> # Show some methods for class "ts"
> matrix(methods(class="ts")[3:8], ncol=2)
> # "tsp" attribute specifies the date-time index
> attributes(timeser)
> # Extract the index
> tail(time(timeser), 11)
> # The index is equally spaced
> diff(tail(time(timeser), 11))
> # Subset the time series
> window(timeser, start=1992, end=1992.25)
```

Plotting *ts* Time Series Objects

The method `plot.ts()` plots *ts* time series objects.

```
> # Create plot
> plot(timeser, type="l", col="red", lty="solid",
+      xlab="", ylab="")
> title(main="Brownian Motion", line=1)  ##  Add title
```



EuStockMarkets Data

R includes a number of base packages that are already installed and loaded.

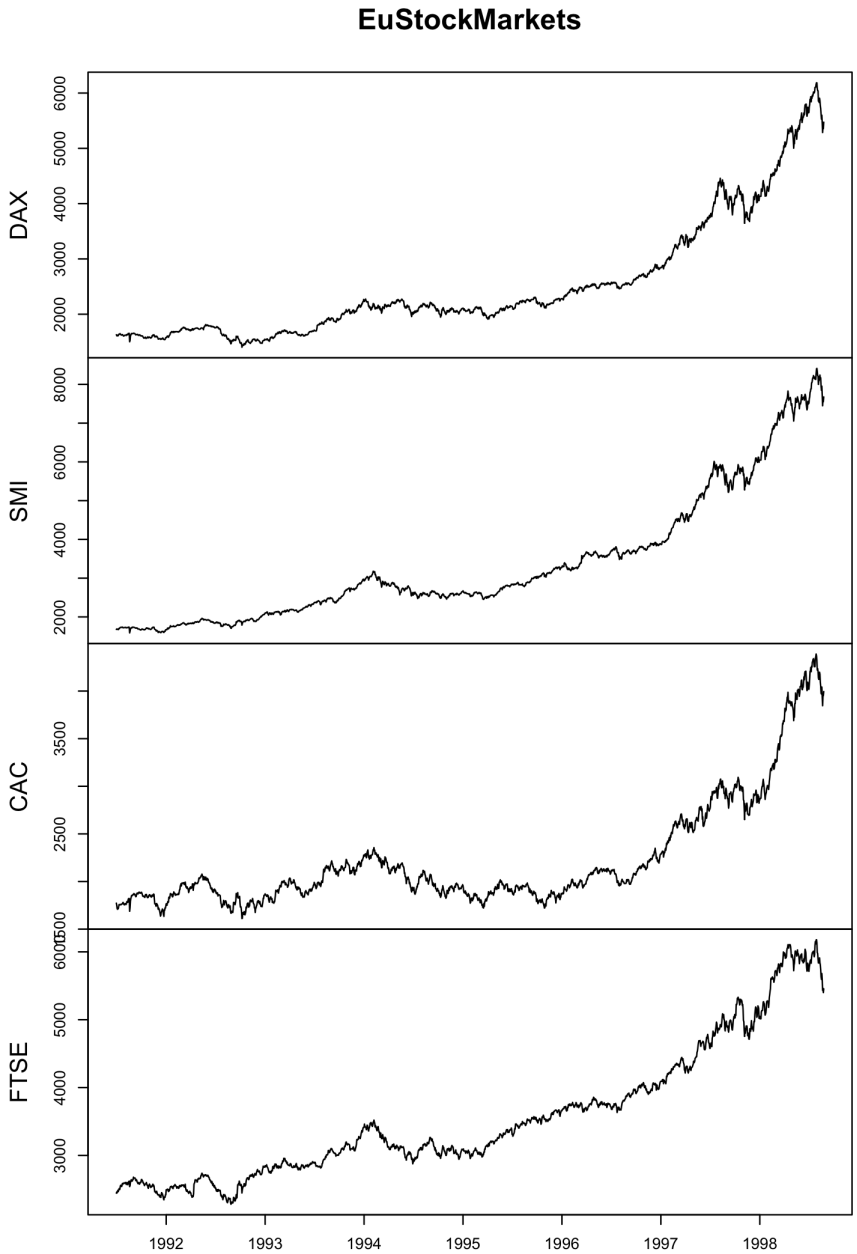
datasets is a base package containing various datasets, for example: EuStockMarkets.

The EuStockMarkets dataset contains daily closing prices of european stock indices.

EuStockMarkets is a mts() time series object.

The EuStockMarkets *date-time* index is equally spaced (*regular*), so the *year-fraction* dates don't correspond to actual trading days.

```
> class(EuStockMarkets)  ##  Multiple ts object
> dim(EuStockMarkets)
> head(EuStockMarkets, 3)  ##  Get first three rows
> # EuStockMarkets index is equally spaced
> diff(tail(time(EuStockMarkets), 11))
> # Plot all the columns in separate panels
> plot(EuStockMarkets, main="EuStockMarkets", xlab="")
```

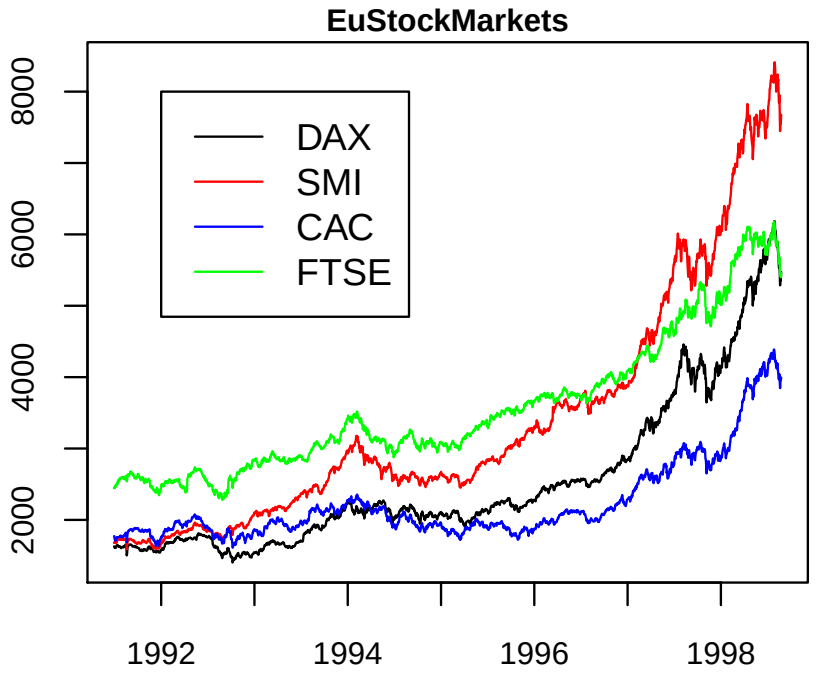


Plotting EuStockMarkets Data

The argument `plot.type="single"` for method `plot.zoo()` allows plotting multiple lines in a single panel (pane).

The four `EuStockMarkets` time series can be plotted in a single panel (pane).

```
> # Plot in single panel
> plot(EuStockMarkets, main="EuStockMarkets",
+      xlab="", ylab="", plot.type="single",
+      col=c("black", "red", "blue", "green"))
> # Add legend
> legend(x=1992, y=8000,
+  legend=colnames(EuStockMarkets),
+  col=c("black", "red", "blue", "green"),
+  lwd=6, lty=1)
```



zoo Time Series Objects

The package *zoo* is designed for managing *irregular* time series and ordered objects of class *zoo*.

Irregular time series have *date-time* indices that aren't equally spaced (because of weekends, overnight hours, etc.).

The function `zoo()` creates a *zoo* object from a `numeric` vector or matrix, and an associated *date-time* index.

The *zoo* index is a vector of *date-time* objects, and can be from any *date-time* class.

The *zoo* class can manage *irregular* time series whose *date-time* index isn't equally spaced.

```
> library(zoo)  ##  Load package zoo
> # Create zoo time series of random returns
> datev <- Sys.Date() + 0:11
> zoots <- zoo(rnorm(NROW(datev)), order.by=datev)
> zoots
2025-12-01 2025-12-02 2025-12-03 2025-12-04 2025-12-05 2025-12-06 20
    0.1450     0.4383     0.1532     1.0849     1.9995    -0.8119
2025-12-08 2025-12-09 2025-12-10 2025-12-11 2025-12-12
    0.5859     0.3601    -0.0253     0.1509     0.1101
> attributes(zoots)
$index
 [1] "2025-12-01" "2025-12-02" "2025-12-03" "2025-12-04" "2025-12-05
 [6] "2025-12-06" "2025-12-07" "2025-12-08" "2025-12-09" "2025-12-10
[11] "2025-12-11" "2025-12-12"

$class
[1] "zoo"
> class(zoots)  ##  Class "zoo"
[1] "zoo"
> tail(zoots, 3)  ##  Get last few elements
2025-12-10 2025-12-11 2025-12-12
   -0.0253     0.1509     0.1101
```

Operations on *zoo* Time Series

The function `zoo::coredata()` extracts the data contained in *zoo* object, and returns a vector or matrix.

The function `zoo::index()` extracts the time index of a *zoo* object.

The function `xts::.index()` extracts the time index expressed in the number of seconds.

The functions `start()` and `end()` return the time index values of the first and last elements of a *zoo* object.

The functions `cumsum()`, `cummax()`, and `cummin()` return cumulative sums, minima and maxima of a *zoo* object.

```
> zoo::coredata(zoots)  ##  Extract coredata
 [1]  0.1450  0.4383  0.1532  1.0849  1.9995 -0.8119  0.1603  0.5859
[10] -0.0253  0.1509  0.1101
> zoo::index(zoots)  ##  Extract time index
 [1] "2025-12-01" "2025-12-02" "2025-12-03" "2025-12-04" "2025-12-05
 [6] "2025-12-06" "2025-12-07" "2025-12-08" "2025-12-09" "2025-12-10
[11] "2025-12-11" "2025-12-12"
> start(zoots)  ##  First date
[1] "2025-12-01"
> end(zoots)  ##  Last date
[1] "2025-12-12"
> zoots[start(zoots)]  ##  First element
2025-12-01
     0.145
> zoots[end(zoots)]  ##  Last element
2025-12-12
      0.11
> zoo::coredata(zoots) <- rep(1, NROW(zoots))  ##  Replace coredata
> cumsum(zoots)  ##  Cumulative sum
2025-12-01 2025-12-02 2025-12-03 2025-12-04 2025-12-05 2025-12-06 20
         1          2          3          4          5          6
2025-12-08 2025-12-09 2025-12-10 2025-12-11 2025-12-12
         8          9         10         11         12
> cummax(cumsum(zoots))
2025-12-01 2025-12-02 2025-12-03 2025-12-04 2025-12-05 2025-12-06 20
         1          2          3          4          5          6
2025-12-08 2025-12-09 2025-12-10 2025-12-11 2025-12-12
         8          9         10         11         12
> cummin(cumsum(zoots))
2025-12-01 2025-12-02 2025-12-03 2025-12-04 2025-12-05 2025-12-06 20
         1          1          1          1          1          1
2025-12-08 2025-12-09 2025-12-10 2025-12-11 2025-12-12
         1          1          1          1          1
```

Single Column *zoo* Time Series

Single column *zoo* time series usually don't have a dimension attribute (they have a NULL dimension), and they don't have a column name, unlike multi-column *zoo* time series.

Single column *zoo* time series without a dimension attribute should be avoided, since they can cause hard to detect bugs.

If a single column *zoo* time series is created from a single column matrices, then it have a dimension attribute, and can be assigned a column name.

```
> zoots <- zoo(matrix(cumsum(rnorm(10)), nc=1),
+   order.by=seq(from=as.Date("2013-06-15"), by="day", len=10))
> colnames(zoots) <- "zoots"
> tail(zoots)
           zoots
2013-06-19  2.63
2013-06-20  2.11
2013-06-21  2.24
2013-06-22  2.08
2013-06-23  2.15
2013-06-24  2.08
> dim(zoots)
[1] 10  1
> attributes(zoots)
$dim
[1] 10  1

$index
 [1] "2013-06-15" "2013-06-16" "2013-06-17" "2013-06-18" "2013-06-19
 [6] "2013-06-20" "2013-06-21" "2013-06-22" "2013-06-23" "2013-06-24

$class
[1] "zoo"

$dimnames
$dimnames[[1]]
NULL

$dimnames[[2]]
[1] "zoots"
```

The lag() and diff() Functions

The method `lag.zoo()` returns a lagged version of a *zoo* time series, shifting the time index by `"k"` observations.

If `"k"` is positive, then `lag.zoo()` shifts values from the future to the present, and if `"k"` is negative then it shifts them from the past.

This is the opposite of what is usually considered as a positive *lag*.

A positive *lag* should replace the current value with values from the past (negative lags should replace with values from the future).

The method `diff.zoo()` returns the difference between a *zoo* time series and its proper lagged version from the past, given a positive *lag* value.

By default, the methods `lag.zoo()` and `diff.zoo()` omit any `NA` values they may have produced, and return shorter time series.

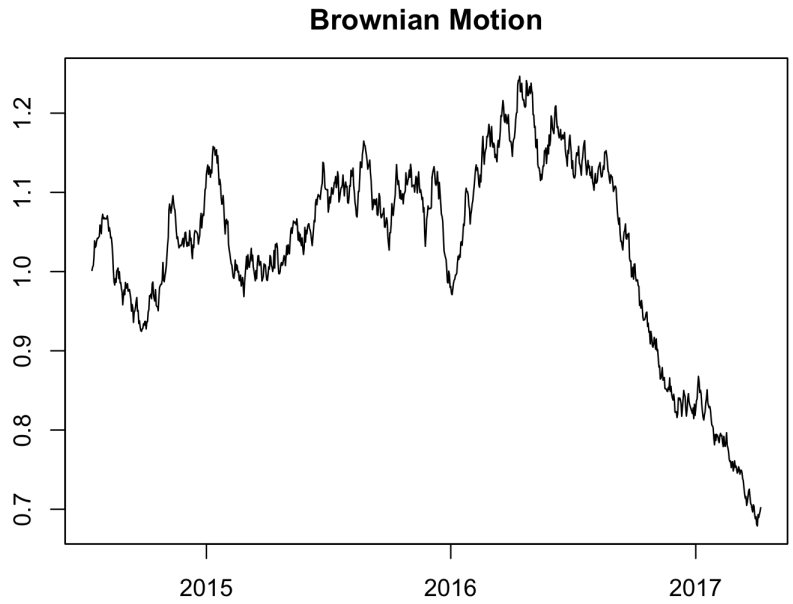
If the `"na.pad"` argument is set to `TRUE`, then they return time series of the same length, with `NA` values added where needed.

```
> zoo::coredata(zoots) <- (1:10)^2  ##  Replace coredata
> zoots
           zoots
2013-06-15     1
2013-06-16     4
2013-06-17     9
2013-06-18    16
2013-06-19    25
2013-06-20    36
2013-06-21    49
2013-06-22    64
2013-06-23    81
2013-06-24   100
> lag(zoots)  ##  One day lag
           zoots
2013-06-15     4
2013-06-16     9
2013-06-17    16
2013-06-18    25
2013-06-19    36
2013-06-20    49
2013-06-21    64
2013-06-22    81
2013-06-23   100
> lag(zoots, 2)  ##  Two day lag
           zoots
2013-06-15     9
2013-06-16    16
2013-06-17    25
2013-06-18    36
2013-06-19    49
2013-06-20    64
2013-06-21    81
2013-06-22   100
> lag(zoots, k=-1)  ##  Proper one day lag
           zoots
2013-06-16     1
2013-06-17     4
```

Plotting *zoo* Time Series

zoo time series can be plotted using the generic function `plot()`, which dispatches the `plot.zoo()` method.

```
> # Initialize the random number generator
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> library(zoo)  ##  Load package zoo
> # Create index of daily dates
> datev <- seq(from=as.Date("2014-07-14"), by="day", length.out=100(
> # Create vector of geometric Brownian motion
> datav <- exp(cumsum(rnorm(NROW(datev))/100))
> # Create zoo series of geometric Brownian motion
> zoots <- zoo(x=datav, order.by=datev)
> # Plot using method plot.zoo()
> plot.zoo(zoots, xlab="", ylab="")
> title(main="Brownian Motion", line=1)  ##  Add title
```



Subsetting *zoo* Time Series

zoo time series can be subset in similar ways to `matrices` and *ts* time series.

The function `window()` can also subset *zoo* time series objects.

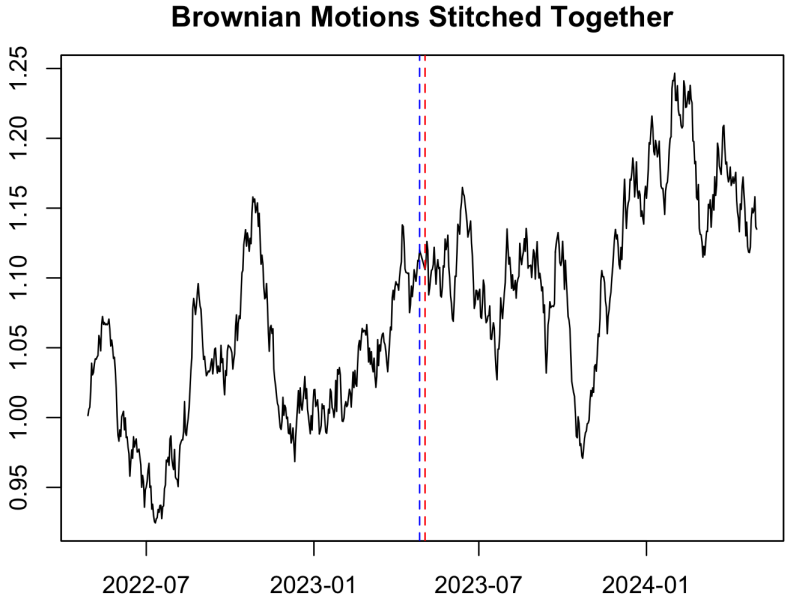
In addition, *zoo* time series can be subset using `Date` objects.

```
> # Subset zoo as matrix
> zoots[459:463, 1]
> # Subset zoo using window()
> window(zoots,
+  start=as.Date("2014-10-15"),
+  end=as.Date("2014-10-19"))
> # Subset zoo using Date object
> zoots[as.Date("2014-10-15")]
```

Sequential Joining *zoo* Time Series

The *zoo* time series can be joined sequentially using function `rbind()`.

```
> # Initialize the random number generator
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> library(zoo)  ##  Load package zoo
> # Create daily date series of class "Date"
> tday <- Sys.Date()
> index1 <- seq(tday-2*365, by="days", length.out=365)
> # Create zoo time series of random returns
> zoo1 <- zoo(rnorm(NROW(index1)), order.by=index1)
> # Create another zoo time series of random returns
> index2 <- seq(tday-360, by="days", length.out=365)
> zoo2 <- zoo(rnorm(NROW(index2)), order.by=index2)
> # rbind the two time series - ts1 supersedes ts2
> zooub2 <- zoo2[zoo::index(zoo2) > end(zoo1)]
> zoo3 <- rbind(zoo1, zooub2)
> # Plot zoo time series of geometric Brownian motion
> plot(exp(cumsum(zoo3)/100), xlab="", ylab="")
> # Add vertical lines at stitch point
> abline(v=end(zoo1), col="blue", lty="dashed")
> abline(v=start(zoo2), col="red", lty="dashed")
> title(main="Brownian Motions Stitched Together", line=1)  ##  Add title
```



Merging *zoo* Time Series

zoo time series can be combined concurrently by joining their columns using function `merge()`.

Function `merge()` is similar to function `cbind()`.

If the `all=TRUE` option is set, then `merge()` returns the union of their dates, otherwise it returns their intersection.

The `merge()` operation can produce `NA` values.

```
> # Create daily date series of class "Date"
> index1 <- Sys.Date() + -3:1
> # Create zoo time series of random returns
> zoo1 <- zoo(rnorm(NROW(index1)), order.by=index1)
> # Create another zoo time series of random returns
> index2 <- Sys.Date() + -1:3
> zoo2 <- zoo(rnorm(NROW(index2)), order.by=index2)
> merge(zoo1, zoo2)  ##  union of dates
> # Intersection of dates
> merge(zoo1, zoo2, all=FALSE)
```

Managing NA Values

Binding two time series that don't share the same time index produces NA values.

There are two dedicated functions for managing NA values in time series:

- stats::na.omit() removes whole rows of data containing NA values.
- zoo::na.locf() replaces NA values with the most recent non-NA values prior to it (*locf* stands for *last observation carry forward*).

Copying the last non-NA values forward causes less data loss than removing whole rows of data.

na.locf() with argument fromLast=TRUE operates in reverse order, starting from the end.

But copying values forward requires initializing the first row of data, to guarantee that initial NA values are also over-written.

The initial NA *prices* can be initialized to the first non-NA price in the future, which can be done by calling zoo::na.locf() with the argument fromLast=TRUE.

But the initial NA values in *returns* data should be initialized to *zero*, without carrying data backward from the future, to avoid data *snooping*.

```
> # Create matrix containing NA values
> matv <- sample(18)
> matv[sample(NROW(matv), 4)] <- NA
> matv <- matrix(matv, nc=3)
> # Replace NA values with most recent non-NA values
> zoo::na.locf(matv)
> # Get time series of prices
> pricev <- mget(c("VTI", "VXX"), envir=rutils::etfenv)
> pricev <- lapply(pricev, quantmod::Cl)
> pricev <- rutils::do_call(cbind, pricev)
> sum(is.na(pricev))
> # Carry forward and backward non-NA prices
> pricev <- zoo::na.locf(pricev, na.rm=FALSE)
> pricev <- zoo::na.locf(pricev, na.rm=FALSE, fromLast=TRUE)
> sum(is.na(pricev))
> # Remove whole rows containing NA returns
> retp <- rutils::etfenv$returns
> sum(is.na(retp))
> retp <- na.omit(retp)
> # Or carry forward non-NA returns (preferred)
> retp <- rutils::etfenv$returns
> retp[1, is.na(retp[1, ])] <- 0
> retp <- zoo::na.locf(retp, na.rm=FALSE)
> sum(is.na(retp))
```

Managing NA Values in "xts" Time Series

The function `na.locf.xts()` from package *xts* is faster than `zoo::na.locf()`, but it only operates on time series of class `"xts"`.

```
> # Replace NAs in xts time series
> pricev <- rutils::etfenv$prices[, 1]
> head(pricev)
> sum(is.na(pricev))
> library(quantmod)
> pricezoo <- zoo::na.locf(pricev, na.rm=FALSE, fromLast=TRUE)
> pricexts <- xts:::na.locf.xts(pricev, fromLast=TRUE)
> all.equal(pricezoo, pricexts, check.attributes=FALSE)
> library(microbenchmark)
> summary(microbenchmark(
+   zoo=zoo::na.locf(pricev, fromLast=TRUE),
+   xts=xts:::na.locf.xts(pricev, fromLast=TRUE),
+   times=10))[, c(1, 4, 5)]  ##  end microbenchmark summary
```

Coercing Time Series Objects Into *zoo*

The generic function `as.zoo()` coerces objects into *zoo* time series.

The function `as.zoo()` creates a *zoo* object with a `numeric` *date-time* index, with *date-time* encoded as a *year-fraction*.

The *year-fraction* can be *approximately* converted to a `Date` object by first calculating the number of days since the *epoch* (1970), and then coercing the `numeric` days using `as.Date()`.

The function `date_decimal()` from package *lubridate* converts `numeric` *year-fraction* dates into `POSIXct` objects.

The function `date_decimal()` provides a more accurate way of converting a *year-fraction* index to `POSIXct`.

```
> class(EuStockMarkets)  ##  Multiple ts object
> # Coerce mts object into zoo
> zoots <- as.zoo(EuStockMarkets)
> class(zoo::index(zoots))  ##  Index is numeric
> head(zoots, 3)
> # Approximately convert index into class "Date"
> zoo::index(zoots) <- as.Date(365*(zoo::index(zoots)-1970))
> head(zoots, 3)
> # Convert index into class "POSIXct"
> zoots <- as.zoo(EuStockMarkets)
> zoo::index(zoots) <- date_decimal(zoo::index(zoots))
> head(zoots, 3)
```

Coercing *zoo* Time Series Into Class *ts*

The generic function `as.ts()` from package *stats* coerces time series objects (including *zoo*) into *ts* time series.

The function `as.ts()` creates a *ts* object with a `frequency=1`, implying a *"day"* time unit, instead of a *"year"* time unit suitable for *year-fraction* dates.

A *ts* time series can be created from a *zoo* using the function `ts()`, after extracting the data and date attributes from *zoo*.

The function `decimal_date()` from package *lubridate* converts `POSIXct` objects into `numeric` *year-fraction* dates.

```
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> # Create index of daily dates
> datev <- seq(from=as.Date("2014-07-14"), by="day", length.out=1000
> # Create vector of geometric Brownian motion
> datav <- exp(cumsum(rnorm(NROW(datev))/100))
> # Create zoo time series of geometric Brownian motion
> zoots <- zoo(x=datav, order.by=datev)
> head(zoots, 3)  ##  zoo object
> # as.ts() creates ts object with frequency=1
> timeser <- as.ts(zoots)
> tsp(timeser)  ##  Frequency=1
> # Get start and end dates of zoots
> startd <- decimal_date(start(zoots))
> endd <- decimal_date(end(zoots))
> # Calculate frequency of zoots
> tstep <- NROW(zoots)/(endd-startd)
> datav <- zoo::coredata(zoots)  ##  Extract data from zoots
> # Create ts object using ts()
> timeser <- ts(data=datav, start=startd, frequency=tstep)
> # Display start of time series
> window(timeser, start=start(timeser), end=start(timeser)+4/365)
> head(time(timeser))  ##  Display index dates
> head(as.Date(date_decimal(zoo::coredata(time(timeser)))))
```

Coercing Irregular Time Series Into Class *ts*

Irregular time series cannot be properly coerced into *ts* time series without modifying their index.

The function `as.ts()` creates `NA` values when it coerces irregular time series into a *ts* time series.

```
> # Create weekday Boolean vector
> wkdays <- weekdays(zoo::index(zoots))
> wkdayl <- !((wkdays == "Saturday") | (wkdays == "Sunday"))
> # Remove weekends from zoo time series
> zoots <- zoots[wkdayl, ]
> head(zoots, 7)  ##  zoo object
> # as.ts() creates NA values
> timeser <- as.ts(zoots)
> head(timeser, 7)
> # Create vector of regular dates, including weekends
> datev <- seq(from=start(zoots), by="day", length.out=NROW(zoots))
> zoo::index(zoots) <- datev
> timeser <- as.ts(zoots)
> head(timeser, 7)
```

Class *xts* Time Series Objects

The package *xts* defines time series objects of class *xts*,

- Class *xts* is an extension of the *zoo* class (derived from *zoo*),
- Class *xts* is the most widely accepted time series class,
- Class *xts* is designed for high-frequency and *OHLC* data,
- Class *xts* contains many convenient functions for plotting, calculating rolling max, min, etc.

The function `xts()` creates a *xts* object from a `numeric` vector or matrix, and an associated *date-time* index.

The *xts* index is a vector of *date-time* objects, and can be from any *date-time* class.

The *xts* class can manage *irregular* time series whose *date-time* index isn't equally spaced.

```
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> library(xts)  ##  Load package xts
> # Create xts time series of random returns
> datev <- Sys.Date() + 0:3
> xtsv <- xts(rnorm(NROW(datev)), order.by=datev)
> names(xtsv) <- "random"
> xtsv
> tail(xtsv, 3)  ##  Get last few elements
> first(xtsv)  ##  Get first element
> last(xtsv)  ##  Get last element
> class(xtsv)  ##  Class "xts"
> attributes(xtsv)
> # Get the time zone of an xts object
> tzone(xtsv)
```

Coercing *zoo* Time Series Into Class *xts*

The function `as.xts()` coerces *zoo* time series into *xts* series.

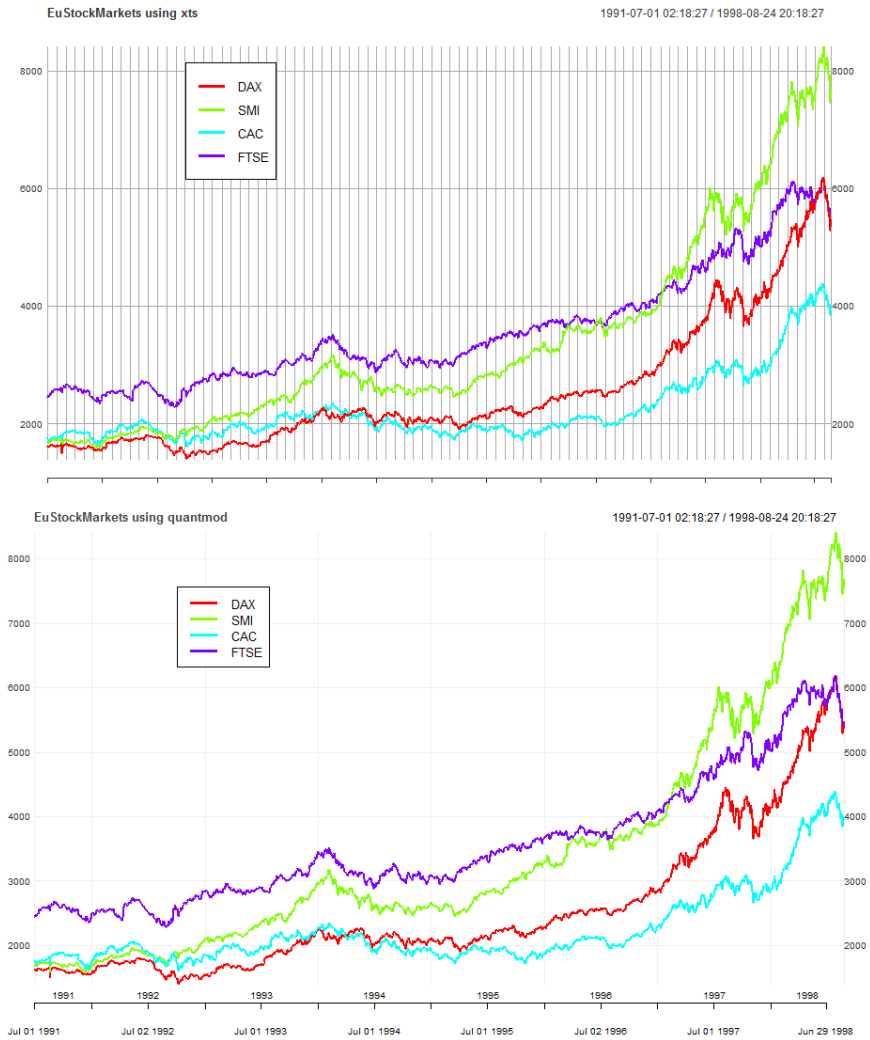
`as.xts()` preserves the *index* attributes of the original time series.

xts can be plotted using the generic function `plot()`, which dispatches the `plot.xts()` method.

```
> load(file="/Users/jerzy/Develop/lecture_slides/data/zoo_data.RData
> class(zoo_stx)
> # as.xts() coerces zoo series into xts series
> library(xts)  ##  Load package xts
> pricexts <- as.xts(zoo_stx)
> dim(pricexts)
> head(pricexts[, 1:4], 4)
> # Plot using plot.xts method
> xts::plot.xts(pricexts[, "Close"], xlab="", ylab="", main="")
> title(main="Stock Prices")  ##  Add title
```

Plotting Multiple *xts* Using Packages *xts* and *quantmod*

```
> library(lubridate)  ##  Load lubridate
> # Coerce EuStockMarkets into class xts
> xtsv <- xts(zoo::coredata(EuStockMarkets),
+       order.by=date_decimal(zoo::index(EuStockMarkets)))
> # Plot all columns in single panel: xts v.0.9-8
> colorv <- rainbow(NCOL(xtsv))
> plot(xtsv, main="EuStockMarkets using xts",
+      col=colorv, major.ticks="years",
+      minor.ticks=FALSE)
> legend("topleft", legend=colnames(EuStockMarkets),
+  inset=0.2, cex=0.7, , lty=rep(1, NCOL(xtsv)),
+  lwd=3, col=colorv, bg="white")
> # Plot only first column: xts v.0.9-7
> plot(xtsv[, 1], main="EuStockMarkets using xts",
+      col=colorv[1], major.ticks="years",
+      minor.ticks=FALSE)
> # Plot remaining columns
> for (colnum in 2:NCOL(xtsv))
+   lines(xtsv[, colnum], col=colorv[colnum])
> # Plot using quantmod
> library(quantmod)
> plotheme <- chart_theme()
> plotheme$col$line.col <- colors
> chart_Series(x=xtsv, theme=plotheme,
+        name="EuStockMarkets using quantmod")
> legend("topleft", legend=colnames(EuStockMarkets),
+  inset=0.2, cex=0.7, , lty=rep(1, NCOL(xtsv)),
+  lwd=3, col=colorv, bg="white")
```



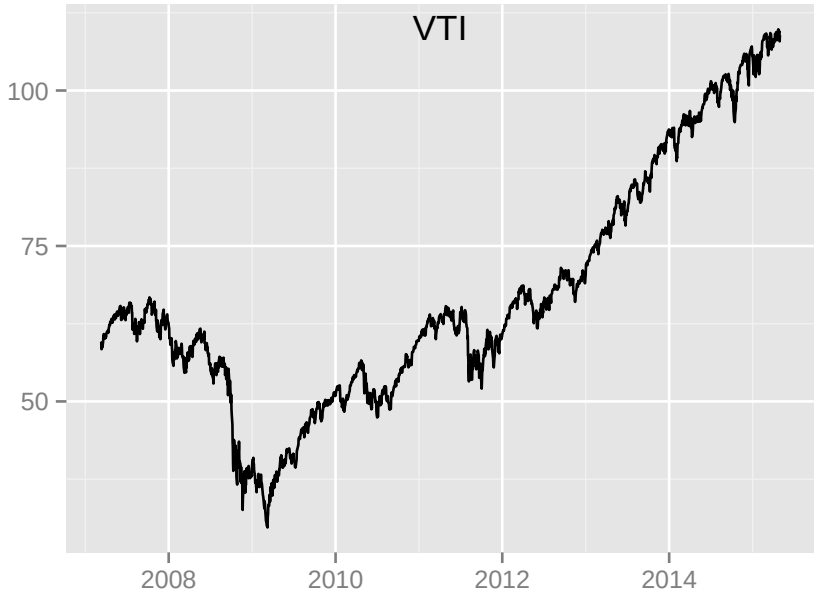
Plotting *xts* Using Package *ggplot2*

xts time series can be plotted using the package *ggplot2*.

The function `qplot()` is the simplest function in the *ggplot2* package, and allows creating line and bar plots.

The function `theme()` customizes plot objects.

```
> library(ggplot2)
> pricev <- rutils::etfenv$prices[, 1]
> pricev <- na.omit(pricev)
> # Create ggplot object
> plotobj <- qplot(x=zoo::index(pricev),
+           y=as.numeric(pricev),
+           geom="line",
+           main=names(pricev)) +
+   xlab("") + ylab("") +
+   theme(  ##  Add legend and title
+     legend.position=c(0.1, 0.5),
+     plot.title=element_text(vjust=-2.0),
+     plot.background=element_blank()
+   )  ##  end theme
> # Render ggplot object
> plotobj
```

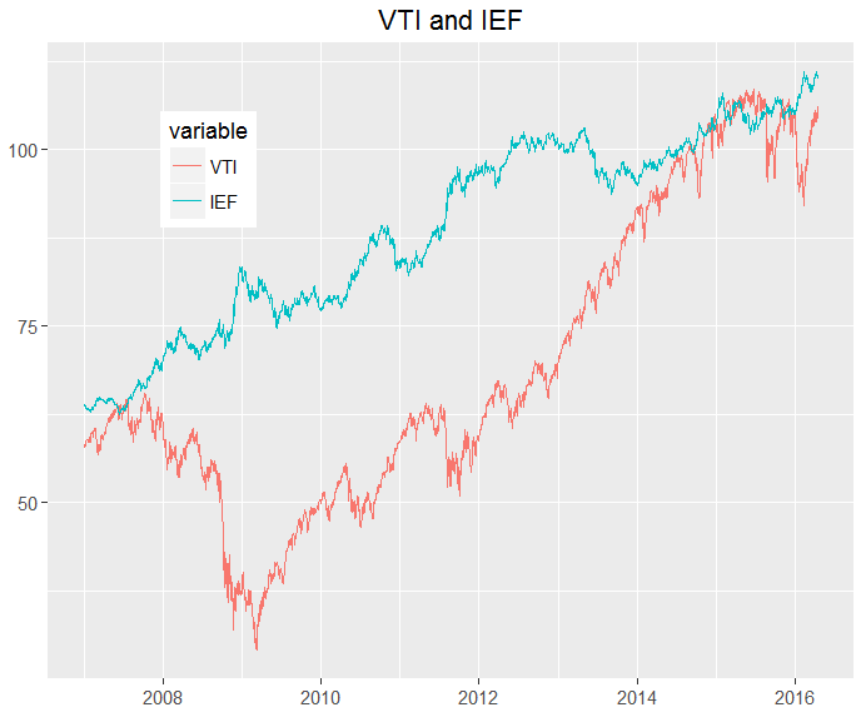


Plotting Multiple *xts* Using Package *ggplot2*

Multiple *xts* time series can be plotted using the function `ggplot()` from package *ggplot2*.

But *ggplot2* functions don't accept time series objects, so time series must be first coerced into `data frames`.

```
> library(rutils)  ##  Load xts time series data
> library(reshape2)
> library(ggplot2)
> pricev <- rutils::etfenv$prices[, c("VTI", "IEF")]
> pricev <- na.omit(pricev)
> # Create data frame of time series
> dframe <- data.frame(datev=zoo::index(pricev), zoo::coredata(price
> # reshape data into a single column
> dframe <- reshape2::melt(dframe, id="dates")
> x11(width=6, height=5)  ##  Open plot window
> # ggplot the melted dframe
> ggplot(data=dframe,
+  mapping=aes(x=datev, y=value, colour=variable)) +
+  geom_line() +
+   xlab("") + ylab("") +
+   ggtitle("VTI and IEF") +
+   theme(  ##  Add legend and title
+     legend.position=c(0.2, 0.8),
+     plot.title=element_text(vjust=-2.0)
+   )  ##  end theme
```



Time series with multiple columns must be reshaped into a single column, which can be performed using the function `melt()` from package *reshape2*,

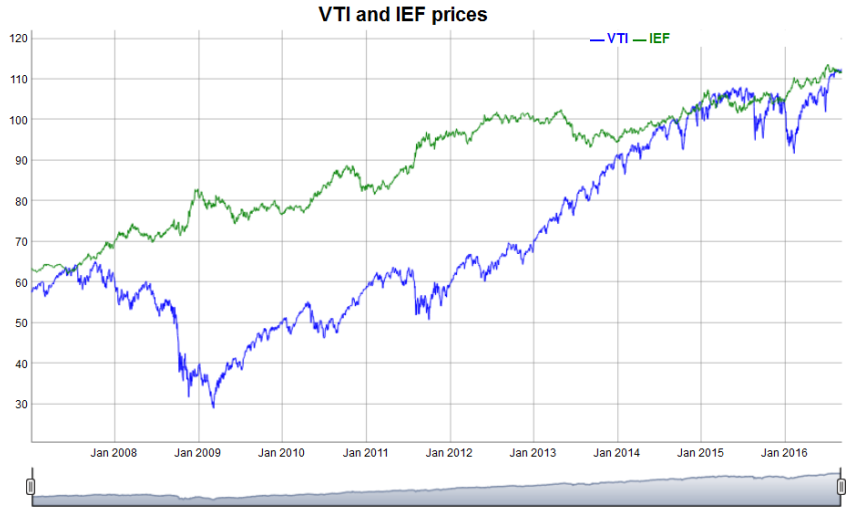
Interactive Time Series Plots Using Package *dygraphs*

The function `dygraph()` from package *dygraphs* creates interactive, zoomable plots from *xts* time series.

The function `dyOptions()` adds options (like colors, etc.) to a *dygraph* plot.

The function `dyRangeSelector()` adds a date range selector to the bottom of a *dygraphs* plot.

```
> # Load rutils which contains etfenv dataset
> library(rutils)
> library(dygraphs)
> pricev <- rutils::etfenv$prices[, c("VTI", "IEF")]
> pricev <- na.omit(pricev)
> # Plot dygraph with date range selector
> dygraph(pricev, main="VTI and IEF prices") %>%
+   dyOptions(colors=c("blue","green")) %>%
+   dyRangeSelector()
```



The *dygraphs* package in `R` is an interface to the *dygraphs JavaScript* charting library.

Interactive *dygraphs* plots require running *JavaScript* code, which can be embedded in *html* documents, and displayed by web browsers.

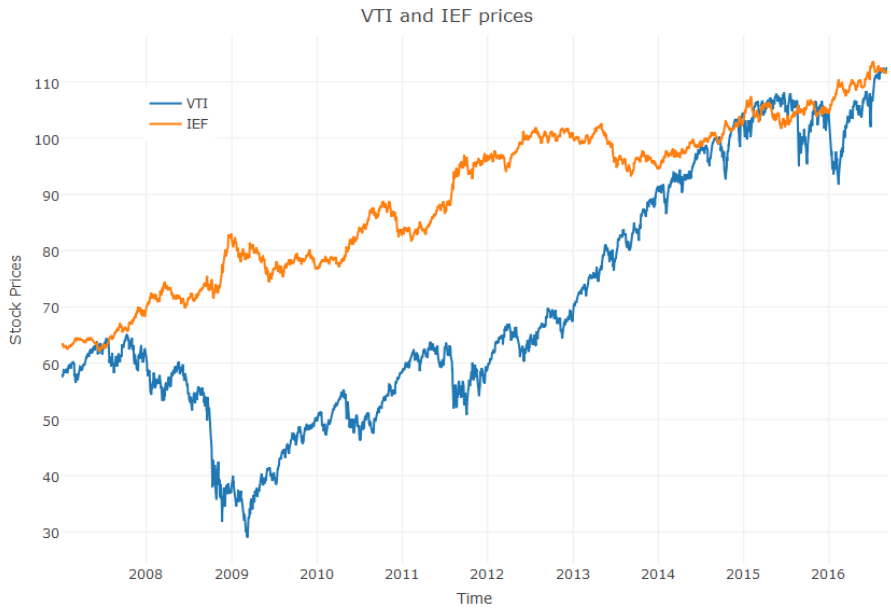
But *pdf* documents can't run *JavaScript* code, so they can't display interactive *dygraphs* plots,

Interactive Time Series Plots Using Package *plotly*

The function `plot_ly()` from package *plotly* creates interactive plots from data residing in `data frames`.

The function `add_trace()` adds elements to a *plotly* plot.

The function `layout()` modifies the layout of a *plotly* plot.




```
> # Load rutils which contains etfenv dataset
> library(rutils)
> library(plotly)
> pricev <- rutils::etfenv$prices[, c("VTI", "IEF")]
> pricev <- na.omit(pricev)
> # Create data frame of time series
> dframe <- data.frame(datev=zoo::index(pricev),
+     zoo::coredata(pricev))
> # Plotly syntax using pipes
> dframe %>%
+   plot_ly(x=~datev, y=~VTI, type="scatter", mode="lines", name="VTI") %>%
+   add_trace(x=~datev, y=~IEF, type="scatter", mode="lines", name="IEF") %>%
+   layout(title="VTI and IEF prices",
+    xaxis=list(title="Time"),
+    yaxis=list(title="Stock Prices"),
+    legend=list(x=0.1, y=0.9))
> # Or use standard plotly syntax
> plotobj <- plot_ly(data=dframe, x=~datev, y=~VTI, type="scatter", mode="lines", name="VTI")
> plotobj <- add_trace(p=plotobj, x=~datev, y=~IEF, type="scatter", mode="lines", name="IEF")
> plotobj <- layout(p=plotobj, title="VTI and IEF prices", xaxis=list(title="Time"), yaxis=list(title="Stock Prices"), legend=list(x=0.
> plotobj
```

Subsetting *xts* Time Series

xts time series can be subset in similar ways as *zoo* time series.

In addition, *xts* time series can be subset using date strings, or date range strings, for example: `["2014-10-15/2015-01-10"]`.

xts time series can be subset by year, week, days, or even seconds.

If only the date is subset, then a comma `","` after the date range isn't necessary.

The function `.subset_xts()` allows fast subsetting of *xts* time series, which for large datasets can be faster than the bracket `"[]"` notation.

```
> # Subset xts using a date range string
> pricev <- rutils::etfenv$prices
> pricesub <- pricev["2014-10-15/2015-01-10", 1:4]
> first(pricesub)
> last(pricesub)
> # Subset Nov 2014 using a date string
> pricesub <- pricev["2014-11", 1:4]
> first(pricesub)
> last(pricesub)
> # Subset all data after Nov 2014
> pricesub <- pricev["2014-11/", 1:4]
> first(pricesub)
> last(pricesub)
> # Comma after date range not necessary
> all.equal(pricev["2014-11", ], pricev["2014-11"])
> # .subset_xts() is faster than the bracket []
> library(microbenchmark)
> summary(microbenchmark(
+   bracket=pricev[10:20, ],
+   subset=xts::.subset_xts(pricev, 10:20),
+   times=10))[, c(1, 4, 5)]
```

Fast Subsetting of *xts* Time Series

Subsetting of *xts* time series can be made much faster if the right operations are used.

Subsetting *xts* time series using `Boolean` vectors is usually faster than using date strings.

But the speed of subsetting can be reduced by additional operations, like coercing strings into dates.

```
> # Specify string representing a date
> datev <- "2014-10-15"
> # Subset prices in two different ways
> pricev <- rutils::etfenv$prices
> all.equal(pricev[zoo::index(pricev) >= datev],
+     pricev[paste0(datev, "/")])
> # Boolean subsetting is slower because coercing string into date
> library(microbenchmark)
> summary(microbenchmark(
+   boolean=(pricev[zoo::index(pricev) >= datev]),
+   date=(pricev[paste0(datev, "/")]),
+   times=10))[, c(1, 4, 5)]  ##  end microbenchmark summary
> # Coerce string into a date
> datev <- as.Date("2014-10-15")
> # Boolean subsetting is faster than using date string
> summary(microbenchmark(
+   boolean=(pricev[zoo::index(pricev) >= datev]),
+   date=(pricev[paste0(datev, "/")]),
+   times=10))[, c(1, 4, 5)]  ##  end microbenchmark summary
```

Subsetting Recurring *xts* Time Intervals

A *recurring time interval* is the same time interval every day, for example the time interval from 9:30AM to 4:00PM every day.

xts series can be subset on recurring time intervals using the `"T"` notation.

For example, to subset the time interval from 9:30AM to 4:00PM every day: `["T09:30:00/T16:00:00"]`

Warning messages that "timezone of object is different than current timezone" can be suppressed by calling the function options() with argument `"xts_check_tz=FALSE"`

```
> pricev <- HighFreq::SPY["2012-04"]
> # Subset recurring time interval using "T notation",
> pricev <- pricev["T10:30:00/T15:00:00"]
> first(pricev["2012-04-16"])  ##  First element of day
> last(pricev["2012-04-16"])  ##  Last element of day
> # Suppress timezone warning messages
> options(xts_check_tz=FALSE)
```

Binding *xts* Time Series by Rows

The function `rbind()` joins the rows of *xts* time series.

If the time series have overlapping time indices then the join produces duplicate rows with the same dates.

The duplicate rows can be removed using the function `duplicated()`.

The function `duplicated()` returns a `Boolean` vector indicating the duplicate elements of a vector.

The function `duplicated()` with argument `"fromLast=TRUE"` identifies duplicate elements starting from the end.

```
> # Create time series with overlapping time indices
> vti1 <- rutils::etfenv$VTI["/2015"]
> vti2 <- rutils::etfenv$VTI["2014/"]
> dates1 <- zoo::index(vti1)
> dates2 <- zoo::index(vti2)
> # Join by rows
> vti <- rbind(vti1, vti2)
> datev <- zoo::index(vti)
> sum(duplicated(datev))
> vti <- vti[!duplicated(datev), ]
> all.equal(vti, rutils::etfenv$VTI)
> # Alternative method - slightly slower
> vti <- rbind(vti1, vti2[!(zoo::index(vti2) %in% zoo::index(vti1))])
> all.equal(vti, rutils::etfenv$VTI)
> # Remove duplicates starting from the end
> vti <- rbind(vti1, vti2)
> vti <- vti[!duplicated(datev), ]
> vtifl <- vti[!duplicated(datev, fromLast=TRUE), ]
> all.equal(vti, vtifl)
```

Properties of *xts* Time Series

xts series always have a `dim` attribute, unlike *zoo*, which have no `dim` attribute when they only have one column of data.

zoo series with multiple columns have a `dim` attribute, and are therefore matrices.

But *zoo* with a single column don't, and are therefore vectors not matrices.

When a *zoo* is subset to a single column, the `dim` attribute is dropped, which can create errors.

```
> pricev <- rutils::etfenv$prices[, c("VTI", "IEF")]
> pricev <- na.omit(pricev)
> str(pricev)  ##  Display structure of xts
> # Subsetting zoo to single column drops dim attribute
> pricezoo <- as.zoo(pricev)
> dim(pricezoo)
> dim(pricezoo[, 1])
> # zoo with single column are vectors not matrices
> c(is.matrix(pricezoo), is.matrix(pricezoo[, 1]))
> # xts always have a dim attribute
> rbind(base=dim(pricev), subs=dim(pricev[, 1]))
> c(is.matrix(pricev), is.matrix(pricev[, 1]))
```

`lag()` and `diff()` Operations on *xts* Time Series

The methods `xts::lag()` and `xts::diff()` for *xts* series differ from those of package *zoo*.

By default, the method `xts::lag()` replaces the current value with values from the past (negative lags replace with values from the future).

The methods `zoo::lag()` and `zoo::diff()` shorten the series by the number of lag periods.

By default, the methods `xts::lag()` and `xts::diff()` retain the same number of elements, by padding with leading or trailing `NA` values.

In order to avoid padding with `NA` values, asset returns can be padded with zeros, and prices can be padded with the first or last elements of the input vector.

```
> # Lag of zoo shortens it by one row
> rbind(base=dim(pricezoo), lag=dim(lag(pricezoo)))
> # Lag of xts doesn't shorten it
> rbind(base=dim(pricev), lag=dim(lag(pricev)))
> # Lag of zoo is in opposite direction from xts
> head(lag(pricezoo, -1), 4)
> head(lag(pricev), 4)
```

Determining Calendar *End points* of *xts* Time Series

The function `endpoints()` from package *xts* extracts the indices of the last observations in each calendar period of time of an *xts* series.

For example:
`endpoints(x, on="hours")`
extracts the indices of the last observations in each hour.

The *end points* calculated by `endpoints()` aren't always equally spaced, and aren't the same as those calculated from fixed intervals.

For example, the last observations in each day aren't equally spaced due to weekends and holidays.

```
> # Indices of last observations in each hour
> endd <- xts::endpoints(pricev, on="hours")
> head(endd)
> # Extract the last observations in each hour
> head(pricev[endd, ])
```

Converting *xts* Time Series to Lower Periodicity

The function `to.period()` converts a time series to a lower periodicity (for example from hourly to daily periodicity).

`to.period()` returns a time series of open, high, low, and close values (*OHLC*) for the lower period.

`to.period()` converts both univariate and *OHLC* time series to a lower periodicity.

```
> # Lower the periodicity to months
> pricem <- to.period(x=pricev, period="months", name="MSFT")
> # Convert colnames to standard OHLC format
> colnames(pricem)
> colnames(pricem) <- sapply(
+   strsplit(colnames(pricem), split=".", fixed=TRUE),
+   function(namev) namev[-1]
+   )  ##  end sapply
> head(pricem, 3)
> # Lower the periodicity to years
> pricey <- to.period(x=pricem, period="years", name="MSFT")
> colnames(pricey) <- sapply(
+   strsplit(colnames(pricey), split=".", fixed=TRUE),
+   function(namev) namev[-1]
+   )  ##  end sapply
> head(pricey)
```

Plotting *OHLC* Time Series Using `chart_Series()`

The function `chart_Series()` from package *quantmod* can plot candlestick plots of *OHLC* prices.

Each *candlestick* displays one period of data, and consists of a box representing the *Open* and *Close* prices, and a vertical line representing the *High* and *Low* prices.

The color of the box signifies whether the *Close* price was higher or lower than the *Open*,

```
> load(file="/Users/jerzy/Develop/lecture_slides/data/zoo_data.RDat
> library(quantmod)  ##  Load package quantmod
> # as.xts() coerces zoo series into xts series
> class(zoo_stx)
> pricexts <- as.xts(zoo_stx)
> dim(pricexts)
> head(pricexts[, 1:4], 4)
> # OHLC candlechart
> plotheme <- chart_theme()
> plotheme$col$up.col <- c("green")
> plotheme$col$dn.col <- c("red")
> chart_Series(x=pricexts["2016-05/2016-06", 1:4], theme=plotheme,
+   name="Candlestick Plot of OHLC Stock Prices")
```

Plotting *OHLC* Time Series Using Package *dygraphs*

The function `dygraph()` from package *dygraphs* creates interactive plots for *xts* time series.

The function `dyCandlestick()` creates a *candlestick* plot object for *OHLC* data, and uses the first four columns to plot *candlesticks*, and it plots any additional columns as lines.

The function `dyOptions()` adds options (like colors, etc.) to a *dygraph* plot.

```
> library(dygraphs)
> # Create dygraphs object
> dyplot <- dygraphs::dygraph(pricexts["2016-05/2016-06", 1:4])
> # Convert dygraphs object to candlestick plot
> dyplot <- dygraphs::dyCandlestick(dyplot)
> # Render candlestick plot
> dyplot
> # Candlestick plot using pipes syntax
> dygraphs::dygraph(pricexts["2016-05/2016-06", 1:4]) %>%
+   dyCandlestick() %>%
+   dyOptions(colors="red", strokeWidth=3)
> # Candlestick plot without using pipes syntax
> dygraphs::dyCandlestick(dygraphs::dyOptions(
+   dygraphs::dygraph(pricexts["2016-05/2016-06", 1:4]),
+   colors="red", strokeWidth=3))
```

Each *candlestick* displays one period of data, and consists of a box representing the *Open* and *Close* prices, and a vertical line representing the *High* and *Low* prices.

The color of the box signifies whether the *Close* price was higher or lower than the *Open*,

Time Series Classes in R

R and other packages contain a number of different time series classes:

- Class *ts* from base package *stats*: native time series class in R, but allows only *regular* (equally spaced) date-time index, not suitable for sophisticated financial applications,
- Class *zoo*: allows *irregular* date-time index, the *zoo* index can be from any *date-time* class,
- Class *xts* extension of *zoo* class: most widely accepted time series class, designed for high-frequency and *OHLC* data, contains convenient functions for plotting, calculating rolling max, min, etc.
- Class *timeSeries* from the *Rmetrics* suite,

```
> # Create zoo time series
> datev <- seq(from=as.Date("2014-07-14"), by="day", length.out=10)
> timeser <- zoo(x=sample(10), order.by=datev)
> class(timeser)
> timeser
> library(xts)
> # Coerce zoo time series to class xts
> pricexts <- as.xts(timeser)
> class(xtseries)
> xtseries
```

Homework Assignment

Required

- Study all the lecture slides in *FRE6871_Lecture_6.pdf*, and run all the code in *FRE6871_Lecture_6.R*